



UNIVERSIDAD MAYOR DE
SAN SIMÓN

FACULTAD DE CIENCIAS Y
TECNOLOGÍA

CARRERA DE INFORMÁTICA



Ingeniería Informática

INFORME FINAL

Gestión de Comandas

Sistema móvil para la gestión de pedidos, ventas,
gastos y reportes financieros en restaurantes

Materia: Programación Móvil

Docente: Dr. Américo Fiorilo Lozada Ph.D.

INTEGRANTES:

Nombre	Carrera	Rol
Acebey Zalazar Jimmy Randall	Ing. Informática	Dev
Cáceres Véliz Erick Alvaro	Ing. Informática	Dev
Revollo Bernal Amilcar Diego	Ing. Informática	SM
Siles Espicho Adrian Eloy	Ing. Informática	Dev
Tinta Mamani Jairo Amado	Ing. Informática	Dev

Carrera: Ing. Informática

Fecha: 05/06/2026

COCHABAMBA — BOLIVIA

Índice general

1. Resumen Ejecutivo	4
1.1. Problema Abordado	4
1.2. Solución Implementada	4
1.3. Arquitectura Técnica	5
1.4. Metodología y Resultado General	5
2. Análisis del Proyecto	6
2.1. Objetivos	6
2.1.1. Objetivo General	6
2.1.2. Objetivos Específicos	6
2.2. Problema o Necesidad Identificada	6
2.3. Contexto	7
2.4. Requisitos Funcionales	7
2.5. Requisitos No Funcionales	8
2.5.1. Historias de Usuario	8
2.5.2. Setup e infraestructura	9
2.5.3. Autenticación y roles	12
2.5.4. Row Level Security	15
2.5.5. Gestión de usuarios (admin)	16
2.5.6. Gestión de mesas (admin)	18
2.5.7. Gestión de menú (admin)	19
2.5.8. Toma de pedidos (mesero)	21
2.5.9. Pagos (admin)	25
2.5.10. Gastos operativos (admin)	26
2.5.11. Vista de mesas en tiempo real (mesero)	28
2.5.12. Llamada al mesero (cliente → mesero)	31
2.5.13. Carta digital pública (web)	32
2.5.14. Vista pública de cliente en mesa (web)	34
2.5.15. Reportes financieros (admin)	36
2.5.16. Historias complementarias implementadas	38

3. Diseño del Proyecto	40
3.1. Stack Tecnológico	40
3.2. Modelo de Datos	40
3.2.1. Entidades y Relaciones	40
3.2.2. Esquema de Tablas Reales	42
3.3. Estructura del Proyecto	44
3.4. Roles y Accesos	45
3.5. Rutas Públicas del Cliente	45
4. Arquitectura del Sistema	46
4.1. Diagrama de Contexto	46
4.2. Diagrama de Contenedores	47
4.3. Tecnologías Utilizadas	48
4.4. Diagrama de Componentes — Funcionalidades Clave	48
4.4.1. Componentes de la aplicación móvil	48
4.4.2. Componentes de rutas web públicas	49
4.4.3. Contexto de Supabase	50
4.4.4. Contenedores de Supabase	51
4.4.5. Componentes de la Base de Datos	51
4.4.6. Vista de Estado de Mesas (Mesero)	52
5. Metodología Aplicada	53
5.1. Metodología de Desarrollo	53
5.2. Ciclo de Vida del Proyecto	53
5.3. Herramientas de Gestión	54
6. Seguimiento y Desarrollo Grupal	55
6.1. Planificación del Trabajo	55
6.2. Roles y Responsabilidades	58
6.3. Control de Versiones	58
6.4. Reporte de Avances	58
6.4.1. Indicadores de Progreso	58
6.4.2. Dificultades Encontradas	60
6.5. Comunicación	61

7. Pruebas y Resultados	62
7.1. Plan de Pruebas	62
7.2. Evidencia de Interfaces y Resultados	62
7.2.1. Acceso al Sistema	62
7.2.2. Flujo de Administrador	63
7.2.3. Flujo de Mesero	67
7.2.4. Flujo de Cocina	68
7.2.5. Flujo de Cliente	69
7.2.6. Control de Versiones	70
7.3. Resultados Obtenidos	71
7.4. Mejoras Propuestas	71
8. Conclusiones	72
8.1. Alcance Logrado	72
8.2. Limitaciones Identificadas	72
8.3. Recomendaciones	73
9. Bibliografía	74
9.1. Tecnologías y Frameworks	74
9.2. Herramientas de Desarrollo y Gestión	74
9.3. Referencias Institucionales	74

1. Resumen Ejecutivo

El proyecto **iComanda** consiste en el desarrollo de una aplicación móvil y web orientada a restaurantes pequeños y medianos que necesitan digitalizar la gestión de comandas, mesas, menú, cocina, pagos, gastos, inventario y reportes operativos. La solución reemplaza el registro manual en papel por un flujo centralizado donde meseros, administradores y cocina trabajan sobre la misma información en tiempo real.

El sistema fue construido con **React Native, Expo Router, TypeScript y Supabase**. La aplicación móvil permite operar desde dispositivos Android, mientras que las rutas públicas web permiten que los clientes consulten la carta digital desde un navegador mediante códigos QR, sin instalar una aplicación ni crear una cuenta tradicional.

1.1 Problema Abordado

En muchos restaurantes pequeños y medianos, los pedidos todavía se registran en papel o mediante mensajes informales entre meseros y cocina. Esto produce errores en los totales, pérdida de comandas, dificultad para conocer el estado real de una mesa y poca visibilidad financiera al final del turno. Además, la administración no cuenta con información inmediata sobre ventas, gastos, platos más vendidos o disponibilidad de insumos.

iComanda aborda esta necesidad mediante una plataforma que integra operación diaria y control administrativo. El mesero registra pedidos desde el celular, cocina ve las órdenes pendientes y las marca como listas, el administrador gestiona menú, usuarios, mesas, pagos QR, gastos, inventario y reportes, y el cliente puede ver la carta o llamar al mesero desde la mesa.

1.2 Solución Implementada

La solución implementada incluye los siguientes módulos principales:

- **Autenticación y roles:** acceso diferenciado para **admin**, **mesero** y **chef** mediante Supabase Auth y perfiles en la tabla **profiles**.
- **Gestión de mesas:** administración de mesas, capacidad, estado activo y generación de enlaces QR por mesa.
- **Gestión de menú:** creación, edición, disponibilidad, imágenes y marcado de platos agotados.
- **Toma de pedidos:** registro de órdenes con mesa, ítems, cantidades, método de pago y cálculo automático del total.
- **Cocina:** panel para que el rol **chef** visualice pedidos en preparación y marque órdenes como **lista**.
- **Tiempo real:** actualización de pedidos, mesas, llamadas del cliente y disponibilidad mediante Supabase Realtime.

- **Carta digital pública:** menú accesible desde navegador por ruta `/menu/[slug]`.
- **Vista pública de cliente en mesa:** ruta `/table/[slug]/[tableId]` para ver el menú y llamar al mesero desde el QR de mesa, sin crear cuenta.
- **Pagos:** soporte para efectivo, QR y tarjeta, con confirmación de pagos QR y registro de `paid_at` para reportes.
- **Gastos e inventario:** registro de egresos, materiales, reposiciones, consumos y control básico de stock.
- **Reportes:** métricas financieras, ranking de platos vendidos y ventas por mesero.

1.3 Arquitectura Técnica

El proyecto utiliza una arquitectura **Backend as a Service (BaaS)**. La aplicación cliente consume directamente los servicios de Supabase, evitando la necesidad de implementar un backend intermedio propio.

- **Frontend móvil y web:** React Native, Expo SDK 54, Expo Router 6.
- **Lenguaje:** TypeScript.
- **Backend administrado:** Supabase con PostgreSQL, Auth, Storage, Realtime, funciones SQL y políticas RLS.
- **Diseño de interfaz:** componentes propios inspirados en Material Design 3.
- **Control de versiones:** Git y GitHub, usando ramas auxiliares de desarrollo que se integraron a la rama principal y luego se eliminaron.

1.4 Metodología y Resultado General

El desarrollo se organizó bajo una metodología ágil basada en Scrum, dividido en **3 sprints de 2 semanas**. Cada sprint entregó incrementos funcionales: fundación técnica y autenticación, operación de pedidos y administración, y funcionalidades finales de tiempo real, carta digital, cocina, inventario y reportes.

Como resultado, se obtuvo una solución funcional que centraliza la operación de un restaurante en una sola plataforma, mejora la trazabilidad de pedidos y pagos, reduce errores del registro manual y proporciona información útil para la toma de decisiones administrativas.

2. Análisis del Proyecto

2.1 Objetivos

2.1.1 Objetivo General

Desarrollar una aplicación móvil y web que permita registrar pedidos, gestionar mesas, pagos, menú, cocina, gastos, inventario y reportes financieros en tiempo real para restaurantes pequeños y medianos.

2.1.2 Objetivos Específicos

1. Permitir a los meseros registrar pedidos y pagos (efectivo o QR) desde dispositivos móviles.
2. Implementar un sistema de gestión de menú con actualización de precios y disponibilidad en tiempo real.
3. Registrar gastos operativos e inventario de materiales del restaurante.
4. Generar reportes automáticos de ingresos brutos, netos, egresos y ranking de platos más vendidos.
5. Implementar acceso diferenciado por rol: **mesero**, **admin** y **chef**, además de rutas públicas para clientes.
6. Notificar a los meseros en tiempo real cuando un pedido está listo para ser entregado.
7. Permitir al rol **chef** visualizar pedidos de cocina, marcarlos como listos y gestionar platos agotados.
8. Proveer al mesero una vista en tiempo real del estado de todas sus mesas.
9. Exponer una carta digital pública accesible por QR para los clientes del restaurante.
10. Permitir al cliente ver el menú completo con disponibilidad real y llamar al mesero desde su dispositivo, sin descargar ninguna app.

2.2 Problema o Necesidad Identificada

Los restaurantes pequeños y medianos continúan registrando pedidos en papel, lo que genera una serie de problemas operacionales recurrentes:

1. **Errores de cálculo en caja:** El cierre diario depende de sumas manuales que son propensas a errores humanos.
2. **Falta de conciliación:** Es difícil comparar ventas registradas con ingresos reales, especialmente cuando se mezclan pagos en efectivo y QR.
3. **Sin visibilidad financiera en tiempo real:** El administrador no cuenta con datos inmediatos sobre el desempeño del día.

4. **Procesos lentos de verificación:** Al final de cada turno, validar cada comanda consume tiempo significativo del personal.
5. **Sin trazabilidad de mesas:** No existe un mecanismo para conocer el estado de cada mesa (libre, ocupada, con pedido listo) sin acercarse físicamente.
6. **Sin canal cliente–mesero:** El cliente debe interrumpir al mesero para solicitar atención, lo que genera fricciones durante el servicio.

2.3 Contexto

El sistema está diseñado para restaurantes de tamaño pequeño a mediano que buscan modernizar su gestión operativa sin inversión en infraestructura de servidores propios. El modelo BaaS (Backend as a Service) sobre Supabase permite desplegar toda la solución sin administrar servidores, escalando automáticamente con la demanda.

El público objetivo comprende:

- **Meseros:** usuarios principales del día a día, con necesidad de interfaces rápidas e intuitivas.
- **Administradores:** responsables de la gestión del menú, gastos y reportes financieros, usuarios, mesas e inventario.
- **Chef/cocina:** responsable de visualizar pedidos en preparación, marcar órdenes listas y controlar platos agotados.
- **Clientes en mesa:** usuarios ocasionales que interactúan mediante una ruta pública por QR, sin necesidad de registro ni descarga de app.

2.4 Requisitos Funcionales

ID	Requisito
RF-01	Registro de pedidos con selección de platillos, mesa y método de pago.
RF-02	Cálculo automático del total por venta.
RF-03	Registro de gastos diarios por parte del administrador.
RF-04	Actualización del menú (precios, disponibilidad, imágenes) en tiempo real.
RF-05	Generación de reportes de ingresos brutos, netos y egresos (diario, semanal, mensual).
RF-06	Reporte de platos más vendidos con ranking por período.
RF-07	Acceso diferenciado: mesero, admin y chef con rutas y permisos propios.
RF-08	Notificaciones en tiempo real al mesero cuando el pedido pasa a estado lista .
RF-09	Vista de estado de mesas para el mesero: pendiente, listo, entregado, libre.

ID	Requisito
RF-10	Carta digital pública accesible mediante QR, sin necesidad de app ni registro.
RF-11	El cliente puede escanear un QR por mesa para abrir una vista pública y ver el menú con disponibilidad real.
RF-12	El cliente puede llamar al mesero desde su dispositivo, generando una notificación en tiempo real al mesero asignado.
RF-13	El chef puede ver pedidos activos y marcarlos como lista .
RF-14	El chef puede marcar platos como agotados o disponibles.
RF-15	El administrador puede registrar materiales y movimientos de inventario.
RF-16	El administrador puede consultar ventas por mesero y liberar mesas después del pago confirmado.

Cuadro 2.1. *Requisitos Funcionales del Sistema*

2.5 Requisitos No Funcionales

ID	Requisito
RNF-01	Interfaz intuitiva con curva de aprendizaje mínima.
RNF-02	Cifrado de datos sensibles (transacciones, sesiones).
RNF-03	Escalable para múltiples sucursales.
RNF-04	Disponibilidad alta con recuperación ante fallos.
RNF-05	Compatible con Android.
RNF-06	Procesamiento de pedidos sin latencia perceptible en horas pico.
RNF-07	Base de código preparada para builds y actualizaciones mediante el ecosistema Expo.
RNF-08	Carta digital accesible desde cualquier dispositivo con navegador web.
RNF-09	La sesión del cliente en mesa no requiere registro ni descarga de aplicación.

Cuadro 2.2. *Requisitos No Funcionales del Sistema*

2.5.1 Historias de Usuario

Las historias de usuario representan los requerimientos funcionales del sistema desde la perspectiva de los distintos actores involucrados, principalmente administradores, meseros, chef/cocina y clientes.

Cada historia describe una necesidad específica del usuario junto con el valor que aporta al sistema, permitiendo organizar el desarrollo mediante iteraciones incrementales.

2.5.2 Setup e infraestructura

HU-01 — Inicializar proyecto Expo con TypeScript y Expo Router

HU-01	Inicializar proyecto Expo con TypeScript y Expo Router		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como desarrollador, quiero tener el proyecto base configurado con TypeScript y Expo Router para comenzar a construir sobre una estructura sólida y predecible desde el primer sprint.			
Criterios de aceptación 1. El proyecto compila y corre en Android sin errores ni advertencias críticas. 2. Expo Router está configurado como sistema de navegación principal con soporte para grupos de rutas. 3. <code>tsconfig.json</code> tiene el modo estricto activado (<code>"strict": true</code>). 4. La estructura de carpetas sigue la convención <code>app/</code> con grupos <code>login</code>, <code>(mesero)</code> y <code>(admin)</code>. 5. Las variables de entorno están definidas en <code>.env</code> y excluidas del repositorio vía <code>.gitignore</code>. 6. El proyecto tiene un <code>README.md</code> con instrucciones mínimas de instalación y ejecución local. 7. El repositorio está inicializado en Git con una rama <code>main</code> protegida.			

Cuadro 2.3. *HU-01: Configuración inicial del proyecto*

HU-02 — Configurar proyecto Supabase y variables de entorno

HU-02	Configurar proyecto Supabase y variables de entorno		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como desarrollador, quiero tener el cliente Supabase conectado al proyecto para poder consumir todos sus servicios desde la aplicación móvil.			
Criterios de aceptación 1. <code>supabase-js</code> v2 está instalado como dependencia del proyecto. 2. <code>lib/supabase.ts</code> expone el cliente autenticado con la <code>SUPABASE_URL</code> y <code>SUPABASE_ANON_KEY</code>. 3. <code>lib/supabase-public.ts</code> expone un cliente separado para carta digital y vista pública de mesa. 4. El archivo <code>.env</code> está incluido en <code>.gitignore</code> y se provee un <code>.env.example</code> con las claves vacías.			

Cuadro 2.4. *HU-02: Configuración del cliente Supabase*

HU-03 — Crear migración de esquema inicial de la base de datos

HU-03	Crear migración de esquema inicial de la base de datos		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como desarrollador, quiero tener todas las tablas del sistema creadas mediante migraciones versionadas para mantener el historial del esquema y poder replicar el entorno en cualquier momento.			
Criterios de aceptación 1. Las migraciones crean las tablas principales: <code>profiles</code>, <code>restaurants</code>, <code>tables</code>, <code>menu_items</code>, <code>orders</code>, <code>order_items</code>, <code>expenses</code>, <code>table_calls</code>, <code>materials</code> y <code>material_stock_movements</code>. 2. Todos los enums del sistema están definidos: <code>rol_usuario</code>, <code>estado_orden</code>, <code>metodo_pago</code> y <code>material_movement_type</code>. 3. La migración se ejecuta sin errores en un proyecto Supabase limpio. 4. El tipo de dato <code>decimal(10,2)</code> se usa para todos los campos monetarios.			

Cuadro 2.5. HU-03: Migración del esquema inicial

HU-04 — Crear constraint de orden activa única por mesa

HU-04	Crear constraint de orden activa única por mesa		
Estado	Completada	Asignado	Equipo
Esfuerzo	1 punto	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero evitar que existan dos órdenes activas en la misma mesa simultáneamente para mantener la integridad del servicio y prevenir conflictos de estado.			
Criterios de aceptación 1. El índice único parcial <code>one_active_order_per_table</code> está creado sobre <code>orders(table_id)</code>. 2. El índice excluye órdenes con <code>estado</code> IN (<code>'entregada'</code>, <code>'cancelada'</code>). 3. Intentar insertar una segunda orden activa en la misma mesa falla con error de constraint a nivel de base de datos. 4. Una vez que la orden pasa a <code>entregada</code> o <code>cancelada</code>, la misma mesa acepta una nueva orden sin errores. 5. El constraint está documentado en la migración correspondiente con un comentario explicativo. 6. El comportamiento es verificable mediante pruebas manuales con datos de prueba.			

Cuadro 2.6. HU-04: Constraint de orden activa única por mesa

HU-05 — Crear trigger set_closed_at al entregar una orden

HU-05		Crear trigger set_closed_at al entregar una orden	
Estado	Completada	Asignado	Equipo
Esfuerzo	1 punto	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero que <code>closed_at</code> se asigne automáticamente al pasar una orden a estado <code>entregada</code> para no depender del cliente y garantizar la integridad del historial financiero.			
Criterios de aceptación 1. La función <code>set_closed_at()</code> y el trigger <code>trg_set_closed_at</code> están creados en la migración <code>004_triggers.sql</code>. 2. Al ejecutar <code>UPDATE orders SET estado='entregada'</code>, el campo <code>closed_at</code> se puebla automáticamente con <code>now()</code>. 3. El trigger es de tipo <code>BEFORE UPDATE</code> para que el valor quede disponible en la misma transacción. 4. Si la orden ya tenía <code>estado='entregada'</code>, una actualización posterior no sobrescribe <code>closed_at</code>. 5. Las órdenes con <code>estado='cancelada'</code> no reciben valor en <code>closed_at</code>. 6. El comportamiento es verificable ejecutando un <code>UPDATE</code> directo en la base de datos y consultando el campo resultante.			

Cuadro 2.7. *HU-05: Trigger de cierre automático de orden*

2.5.3 Autenticación y roles

HU-06 — Implementar pantalla de login para mesero y admin

HU-06	Implementar pantalla de login para mesero y admin		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero o administrador, quiero poder iniciar sesión con mi email y contraseña para acceder al sistema según el rol que me corresponde.			
Criterios de aceptación 1. La pantalla presenta un formulario con campos de email y contraseña. 2. Al autenticarse con credenciales correctas, el sistema carga el perfil desde profiles y redirige a (mesero), (admin) o (chef) según el rol. 3. Si las credenciales son incorrectas, se muestra un mensaje de error visible sin exponer detalles técnicos. 4. El campo de contraseña enmascara el texto por defecto, con opción de mostrarlo. 5. La sesión se persiste localmente para que el usuario no deba iniciar sesión en cada apertura de la app. 6. El botón de envío está deshabilitado mientras se procesa la petición para evitar envíos duplicados. 7. El formulario valida que los campos no estén vacíos antes de llamar a Supabase Auth.			

Cuadro 2.8. *HU-06: Pantalla de inicio de sesión*

HU-07 — Proteger rutas según rol con guards de navegación

HU-07			
Proteger rutas según rol con guards de navegación			
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero que cada usuario acceda únicamente a las rutas correspondientes a su rol para respetar el acceso diferenciado definido en los requisitos funcionales.			
Criterios de aceptación 1. Un usuario con <code>rol=mesero</code> que intenta acceder a rutas (<code>admin</code>) es redirigido a la pantalla de login. 2. Un usuario con <code>rol=admin</code> que intenta acceder a rutas (<code>mesero</code>) es redirigido al área de admin. 3. Un usuario con <code>rol=chef</code> es redirigido al área de cocina (<code>chef</code>). 4. Un usuario no autenticado que intenta acceder a cualquier ruta protegida es redirigido a la pantalla de login. 5. La sesión se recupera del almacenamiento local al reiniciar la app sin requerir nuevo login. 6. El guard utiliza el perfil cargado desde <code>profiles</code> en el contexto de autenticación. 7. Los guards están implementados en <code>app/_layout.tsx</code> y redirigen según el grupo de rutas.			

Cuadro 2.9. *HU-07: Guards de navegación por rol*

HU-08 — Crear trigger `sync_role_to_auth` para sincronizar rol

HU-08	Crear trigger <code>sync_role_to_auth</code> para sincronizar rol		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero que el rol de la tabla <code>profiles</code> se mantenga consistente con Supabase Auth para que el control de acceso pueda resolverse de forma confiable.			
Criterios de aceptación 1. El trigger <code>trg_sync_role</code> es de tipo <code>AFTER INSERT OR UPDATE OF rol</code> sobre la tabla <code>profiles</code>. 2. Al modificar el campo <code>rol</code> en <code>profiles</code>, el metadata interno de <code>auth.users</code> se actualiza automáticamente. 3. La función del trigger usa <code>SECURITY DEFINER</code> para poder escribir en el esquema <code>auth</code>. 4. El nuevo rol queda disponible para la aplicación después de refrescar la sesión o recargar el perfil. 5. La sincronización no afecta otros campos de metadata existentes. 6. El trigger está definido en la migración <code>004_triggers.sql</code> y versionado en el repositorio.			

Cuadro 2.10. HU-08: Trigger de sincronización de rol a Supabase Auth

HU-09 — Implementar cierre de sesión

HU-09	Implementar cierre de sesión		
Estado	Completada	Asignado	Equipo
Esfuerzo	1 punto	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero o administrador, quiero poder cerrar mi sesión desde la aplicación para que nadie más pueda operar el sistema desde mi dispositivo.			
Criterios de aceptación 1. El botón de cierre de sesión está accesible desde la navegación principal de cada rol. 2. Al cerrar sesión, se invoca <code>supabase.auth.signOut()</code> y la sesión local queda destruida. 3. Tras el cierre, el usuario es redirigido a la pantalla de login. 4. Intentar navegar hacia atrás tras cerrar sesión no permite acceder a rutas protegidas. 5. El botón de logout muestra un estado de carga mientras se procesa la petición. 6. La acción no genera errores si la sesión ya había expirado previamente.			

Cuadro 2.11. HU-09: Cierre de sesión

2.5.4 Row Level Security

HU-10 — Implementar RLS para la tabla orders

HU-10	Implementar RLS para la tabla orders		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero que un mesero solo pueda ver y modificar sus propias órdenes, y que el administrador tenga acceso total a todas las órdenes del restaurante.			
Criterios de aceptación 1. RLS está habilitado en la tabla <code>orders</code>. 2. La política <code>mesero_own_orders</code> restringe <code>SELECT</code> al <code>mesero_id = auth.uid()</code>. 3. La política <code>mesero_insert_orders</code> permite <code>INSERT</code> solo si el rol en el JWT es <code>mesero</code>. 4. La política <code>mesero_update_own_orders</code> permite <code>UPDATE</code> solo sobre las órdenes propias del mesero. 5. La política <code>admin_all_orders</code> concede acceso <code>ALL</code> a usuarios con rol <code>admin</code>. 6. Un mesero que intenta leer órdenes de otro mesero recibe un resultado vacío, no un error. 7. Las políticas están definidas en la migración <code>002_rls.sql</code>.			

Cuadro 2.12. HU-10: RLS en la tabla orders

HU-11 — Implementar RLS para expenses, menu_items y table_calls

HU-11	Implementar RLS para expenses, menu_items y table_calls		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero que los gastos solo los pueda insertar el administrador, que el menú sea legible públicamente con las restricciones correctas y que las llamadas al mesero estén controladas por rol.			
Criterios de aceptación 1. La política <code>admin_insert_expenses</code> restringe INSERT en <code>expenses</code> a rol <code>admin</code>. 2. La política <code>public_menu_read</code> permite SELECT en <code>menu_items</code> únicamente sobre ítems con <code>disponible=true</code> sin autenticación. 3. La política <code>cliente_full_menu_read</code> permite SELECT del menú completo a usuarios con rol <code>cliente</code>. 4. La política <code>cliente_insert_table_call</code> permite INSERT en <code>table_calls</code> solo si el <code>table_id</code> del JWT coincide con el de la fila. 5. La política <code>mesero_table_calls</code> permite al mesero ver y actualizar llamadas de las mesas con órdenes activas que le pertenecen. 6. La inserción pública de llamadas queda limitada a los datos de mesa/restaurante validados desde la URL. 7. Todas las políticas están definidas en la migración <code>002_rls.sql</code>.			

Cuadro 2.13. HU-11: RLS en expenses, menu_items y table_calls

2.5.5 Gestión de usuarios (admin)

HU-12 — Listar usuarios del sistema

HU-12	Listar usuarios del sistema		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero ver la lista de meseros y admins registrados para conocer el equipo activo del restaurante y gestionar sus accesos.			
Criterios de aceptación 1. La pantalla muestra nombre, email, rol y estado activo de cada usuario. 2. Solo usuarios con rol <code>admin</code> pueden acceder a esta pantalla. 3. La lista diferencia visualmente entre roles <code>mesero</code> y <code>admin</code>. 4. Los usuarios inactivos aparecen en la lista pero con indicador visual de su estado. 5. La pantalla no expone el hash de contraseña ni datos sensibles de autenticación. 6. Los datos se obtienen desde la tabla <code>profiles</code>, no directamente desde <code>auth.users</code>.			

Cuadro 2.14. HU-12: Listado de usuarios del sistema

HU-13 — Crear nuevo usuario mesero o admin

HU-13	Crear nuevo usuario mesero o admin		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero poder crear cuentas para nuevos integrantes del equipo para que puedan acceder al sistema con el rol correspondiente desde el primer día.			
Criterios de aceptación 1. El formulario incluye campos de nombre, email, rol y contraseña temporal. 2. El usuario se crea en Supabase Auth y simultáneamente en la tabla <code>profiles</code> con el mismo id. 3. El rol queda registrado en <code>profiles.rol</code> y disponible para el contexto de autenticación. 4. El email debe ser único; se muestra error si ya existe en el sistema. 5. El nuevo usuario puede iniciar sesión inmediatamente con las credenciales asignadas. 6. El administrador que crea al usuario permanece en la pantalla de listado sin interrupciones tras confirmar. 7. No es posible crear un usuario sin asignar un rol explícito.			

Cuadro 2.15. HU-13: Creación de nuevos usuarios

HU-14 — Activar y desactivar usuario

HU-14	Activar y desactivar usuario		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero poder desactivar un usuario sin eliminarlo para gestionar bajas temporales del personal sin perder el historial de órdenes asociado.			
Criterios de aceptación 1. La lista de usuarios muestra un toggle o acción para activar/desactivar cada cuenta. 2. Al desactivar, el campo <code>activo=false</code> se actualiza en la tabla <code>profiles</code>. 3. Un usuario inactivo no puede iniciar sesión en el sistema. 4. El historial de órdenes del usuario inactivo se conserva intacto en la base de datos. 5. El administrador recibe confirmación visual tras el cambio de estado. 6. No es posible desactivar la propia cuenta del administrador que está en sesión.			

Cuadro 2.16. HU-14: Activación y desactivación de usuarios

2.5.6 Gestión de mesas (admin)

HU-15 — CRUD de mesas del restaurante

HU-15	CRUD de mesas del restaurante		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero crear, editar y desactivar mesas para reflejar la distribución real del restaurante en el sistema.			
Criterios de aceptación 1. Se puede crear una mesa indicando número y label descriptivo opcional. 2. El número de mesa es único dentro del restaurante; se muestra error si se intenta duplicar. 3. El label de una mesa existente puede editarse sin recrearla. 4. Las mesas pueden activarse y desactivarse; las inactivas no aparecen en las vistas operativas del mesero. 5. Las mesas con órdenes activas no pueden desactivarse hasta que la orden sea cerrada. 6. La lista de mesas muestra número, label y estado (activa/inactiva) de forma clara. 7. Las operaciones de creación y edición son accesibles únicamente para el rol admin.			

Cuadro 2.17. HU-15: CRUD de mesas del restaurante

HU-16 — Gestionar restaurante y su slug

HU-16	Gestionar restaurante y su slug		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero configurar el nombre y slug de mi restaurante para que las URLs públicas de la carta digital y sesiones de cliente sean reconocibles y únicas.			
Criterios de aceptación 1. La pantalla de configuración permite editar el nombre y slug del restaurante. 2. El slug es validado para ser único en la tabla restaurants; se muestra error si ya está en uso. 3. El slug admite solo letras minúsculas, números y guiones; se rechaza cualquier otro carácter. 4. El slug se usa automáticamente en las rutas <code>/menu/[slug]</code> y <code>/table/[slug]/[tableId]</code>. 5. Al cambiar el slug, se muestra una advertencia de que las URLs públicas anteriores dejarán de funcionar. 6. Solo el administrador puede acceder y modificar la configuración del restaurante.			

Cuadro 2.18. HU-16: Configuración del restaurante y slug

2.5.7 Gestión de menú (admin)

HU-17 — Listar ítems del menú con filtro por categoría

HU-17	Listar ítems del menú con filtro por categoría		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero ver todos los platos del menú organizados y con capacidad de filtrar por categoría para tener visibilidad y control del catálogo completo.			
Criterios de aceptación 1. La lista muestra nombre, precio, categoría y estado de disponibilidad de cada ítem. 2. El filtro por categoría reduce la lista a los ítems de la categoría seleccionada. 3. Los ítems no disponibles se diferencian visualmente de los disponibles. 4. La lista incluye tanto ítems disponibles como no disponibles en la vista del admin. 5. Desde cada ítem se puede acceder directamente a su formulario de edición. 6. Solo el administrador puede acceder a esta pantalla.			

Cuadro 2.19. HU-17: Listado de ítems del menú

HU-18 — Crear ítem del menú con imagen

HU-18	Crear ítem del menú con imagen		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero agregar nuevos platos al menú con foto, descripción y precio para mantener el catálogo actualizado y atractivo para los clientes.			
Criterios de aceptación 1. El formulario incluye campos de nombre, descripción, precio, categoría y disponibilidad. 2. La imagen se selecciona desde la galería del dispositivo y se sube al bucket <code>menu-imgs</code> de Supabase Storage. 3. La URL pública del CDN se guarda en el campo <code>imagen_url</code> del ítem creado. 4. El campo precio acepta únicamente valores numéricos positivos con hasta dos decimales. 5. El nuevo ítem aparece de inmediato en el listado del menú tras confirmarse la creación. 6. Si la subida de la imagen falla, el formulario muestra el error sin perder los datos ya ingresados. 7. El ítem se crea con <code>disponible=true</code> por defecto salvo que el admin lo deshabilite explícitamente.			

Cuadro 2.20. HU-18: Creación de ítem del menú

HU-19 — Editar precio y descripción de un ítem

HU-19	Editar precio y descripción de un ítem		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero poder actualizar el precio o la descripción de un plato existente sin tener que eliminarlo y recrearlo.			
Criterios de aceptación 1. El formulario de edición llega pre-poblado con los datos actuales del ítem. 2. Los campos editables incluyen nombre, descripción, precio, categoría e imagen. 3. Al confirmar, el campo <code>updated_at</code> se actualiza automáticamente a <code>now()</code>. 4. El cambio de precio no afecta las órdenes anteriores, ya que el precio se guarda como snapshot en <code>order_items.precio_unitario</code>. 5. El cambio se refleja de inmediato en el listado del menú y en la carta digital pública. 6. Si el usuario cancela la edición, no se persiste ningún cambio en la base de datos.			

Cuadro 2.21. HU-19: Edición de ítem del menú

HU-20 — Activar o desactivar disponibilidad de un ítem

HU-20	Activar o desactivar disponibilidad de un ítem		
Estado	Completada	Asignado	Equipo
Esfuerzo	1 punto	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero marcar un plato como no disponible al instante cuando se agota para que los meseros y clientes no puedan pedirlo.			
Criterios de aceptación 1. El toggle de disponibilidad está visible en el listado de menú sin necesidad de entrar al formulario completo. 2. Al desactivar, el campo <code>disponible=false</code> se actualiza inmediatamente en la base de datos. 3. Un ítem con <code>disponible=false</code> no aparece en la carta pública ni en la vista del mesero al tomar pedidos. 4. Un ítem con <code>disponible=false</code> sí aparece en la vista del menú del cliente en mesa, marcado visualmente como no disponible. 5. La operación puede revertirse en cualquier momento activando el toggle nuevamente. 6. El cambio se refleja en la carta digital sin necesidad de recargar la página.			

Cuadro 2.22. HU-20: Control de disponibilidad de ítem del menú

HU-21 — Eliminar ítem del menú

HU-21	Eliminar ítem del menú		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero eliminar platos que ya no se ofrecen en el restaurante para mantener el menú limpio y libre de entradas obsoletas.			
Criterios de aceptación 1. Se muestra un diálogo de confirmación antes de ejecutar la eliminación. 2. Si el ítem tiene <code>order_items</code> asociados, el sistema lo desactiva (<code>disponible=false</code>) en lugar de eliminarlo, preservando la integridad del historial. 3. Si el ítem no tiene órdenes asociadas, la eliminación procede y el registro se borra de la base de datos. 4. La imagen asociada en Supabase Storage también se elimina cuando el registro se borra físicamente. 5. El ítem desaparece del listado del menú y de la carta digital inmediatamente tras la eliminación. 6. El administrador recibe retroalimentación sobre si el ítem fue eliminado o solo desactivado.			

Cuadro 2.23. HU-21: Eliminación de ítem del menú

2.5.8 Toma de pedidos (mesero)

HU-22 — Ver el menú disponible al tomar un pedido

HU-22	Ver el menú disponible al tomar un pedido		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero ver el menú con los platos disponibles agrupados por categoría para seleccionarlos rápidamente al registrar un pedido.			
Criterios de aceptación 1. La vista muestra únicamente ítems con <code>disponible=true</code>. 2. Los ítems están agrupados por categoría con un encabezado visible por cada grupo. 3. Cada ítem muestra nombre, precio e imagen en miniatura. 4. La vista no muestra el botón de gestión ni opciones de edición. 5. La carga del menú no requiere recargar la pantalla si fue actualizado recientemente. 6. Los datos se obtienen desde el hook <code>useMenu</code> que aplica la política RLS correspondiente.			

Cuadro 2.24. HU-22: Vista del menú disponible para el mesero

HU-23 — Crear una nueva orden seleccionando mesa e ítems

HU-23	Crear una nueva orden seleccionando mesa e ítems		
Estado	Completada	Asignado	Equipo
Esfuerzo	5 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero crear una orden eligiendo una mesa libre y agregando ítems con cantidades para registrar el pedido del cliente de forma rápida y precisa.			
Criterios de aceptación 1. El formulario permite seleccionar únicamente mesas sin órdenes activas. 2. Es posible agregar múltiples ítems con cantidad variable al mismo pedido. 3. Cada ítem permite agregar una nota opcional (ej.: sin picante, sin cebolla). 4. El campo <code>precio_unitario</code> de cada <code>order_item</code> se guarda como snapshot del precio actual, no como referencia dinámica. 5. El <code>total</code> de la orden se calcula automáticamente como suma de los subtotales. 6. La orden se crea con <code>estado='activa'</code> y <code>pago_confirmado=false</code> por defecto cuando requiere confirmación posterior. 7. Si la mesa seleccionada ya tiene una orden activa (race condition), el sistema muestra un error y no crea la orden duplicada. 8. El mesero queda como <code>mesero_id</code> de la orden recién creada.			

Cuadro 2.25. HU-23: Creación de nueva orden

HU-24 — Ver detalle de una orden activa

HU-24	Ver detalle de una orden activa		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero ver los ítems, el total y el estado actual de una orden activa para saber exactamente qué pidió la mesa y en qué punto está.			
Criterios de aceptación 1. La pantalla muestra la lista de <code>order_items</code> con nombre, cantidad, precio unitario y subtotal. 2. El total de la orden es visible y corresponde a la suma de los subtotales. 3. El estado actual de la orden (<code>activa</code>, <code>lista</code>, <code>entregada</code>) es visible de forma destacada. 4. Las notas por ítem se muestran si existen. 5. El método de pago registrado es visible en el detalle. 6. El mesero puede acceder al detalle únicamente de sus propias órdenes activas.			

Cuadro 2.26. HU-24: Detalle de orden activa

HU-25 — Agregar ítems a una orden existente

HU-25	Agregar ítems a una orden existente		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero poder agregar más platos a una orden pendiente si el cliente solicita algo extra durante el servicio.			
Criterios de aceptación 1. La opción de agregar ítems solo está disponible cuando la orden tiene <code>estado='activa'</code>. 2. Los nuevos <code>order_items</code> se insertan con el snapshot del precio vigente al momento de agregar. 3. El total de la orden se recalcula y actualiza automáticamente tras agregar ítems. 4. No es posible agregar ítems a una orden con <code>estado='lista'</code> o superior. 5. La operación es accesible desde el detalle de la orden con un botón claramente identificado. 6. Los nuevos ítems se reflejan inmediatamente en el detalle de la orden sin recargar la pantalla.			

Cuadro 2.27. HU-25: Adición de ítems a una orden existente

HU-26 — Marcar una orden como entregada

HU-26	Marcar una orden como entregada		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero marcar una orden como entregada una vez que llevé los platos a la mesa para liberar el estado de la mesa y cerrar el ciclo del pedido.			
Criterios de aceptación 1. La acción de marcar como entregada solo está disponible cuando la orden tiene <code>estado='lista'</code>. 2. Al confirmar, el estado transiciona de <code>lista</code> a <code>entregada</code>. 3. El trigger <code>trg_set_closed_at</code> asigna <code>closed_at</code> automáticamente en la misma transacción. 4. La mesa pasa a estado libre en la vista del mesero inmediatamente después. 5. La orden entregada ya no aparece en la lista de órdenes activas del mesero. 6. Se muestra un diálogo de confirmación antes de ejecutar la transición para evitar errores.			

Cuadro 2.28. HU-26: Marcado de orden como entregada

HU-27 — Cancelar una orden

HU-27	Cancelar una orden		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero o administrador, quiero poder cancelar una orden por error o solicitud del cliente para que la mesa quede libre y el pedido no se contabilice.			
Criterios de aceptación 1. La cancelación solo es posible cuando la orden tiene <code>estado='activa'</code>. 2. Al confirmar, el estado transiciona a <code>cancelada</code>. 3. La mesa queda libre en la vista del mesero inmediatamente después de la cancelación. 4. Las órdenes canceladas no aparecen en reportes de ingresos. 5. Se muestra un diálogo de confirmación antes de ejecutar la cancelación. 6. El mesero solo puede cancelar sus propias órdenes; el admin puede cancelar cualquiera. 7. La orden cancelada permanece en la base de datos con su historial de ítems intacto.			

Cuadro 2.29. HU-27: Cancelación de orden

HU-28 — Seleccionar método de pago al crear la orden

HU-28	Seleccionar método de pago al crear la orden		
Estado	Completada	Asignado	Equipo
Esfuerzo	1 punto	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero indicar si el cliente pagará en efectivo o por QR al registrar el pedido para facilitar el posterior proceso de cierre de la venta.			
Criterios de aceptación 1. El formulario de nueva orden incluye un selector con las opciones <code>efectivo</code> y <code>qr</code>. 2. El método de pago seleccionado se guarda en <code>orders.metodo_pago</code>. 3. No es posible confirmar la creación de la orden sin seleccionar un método de pago. 4. El método de pago seleccionado es visible en el detalle de la orden. 5. Para órdenes con <code>metodo_pago='efectivo'</code> o <code>tarjeta</code>, el pago puede confirmarse al entregar la orden. 6. Para órdenes con <code>metodo_pago='qr'</code>, <code>pago_confirmado</code> permanece en <code>false</code> hasta que el admin lo confirme.			

Cuadro 2.30. HU-28: Selección de método de pago

2.5.9 Pagos (admin)

HU-29 — Ver órdenes con pago QR pendiente de confirmación

HU-29	Ver órdenes con pago QR pendiente de confirmación		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero ver todas las órdenes con pago QR pendiente de verificación para poder confirmarlos manualmente y que queden registrados en los reportes.			
Criterios de aceptación 1. La lista filtra órdenes con <code>metodo_pago='qr'</code> y <code>pago_confirmado=false</code>. 2. Cada entrada muestra número de mesa, total y hora de creación de la orden. 3. La lista se actualiza en tiempo real cuando una nueva orden QR entra al sistema. 4. Las órdenes ya confirmadas desaparecen de esta lista. 5. Solo el administrador puede acceder a esta pantalla. 6. La pantalla muestra un estado vacío descriptivo cuando no hay pagos QR pendientes.			

Cuadro 2.31. HU-29: Listado de pagos QR pendientes

HU-30 — Confirmar pago QR de una orden

HU-30	Confirmar pago QR de una orden		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero confirmar el pago QR de una orden para que el ingreso quede registrado en los reportes financieros y haya trazabilidad del cobro.			
Criterios de aceptación 1. Al confirmar, <code>orders.pago_confirmado</code> cambia de <code>false</code> a <code>true</code>. 2. La confirmación actualiza la misma fila de <code>orders</code> y registra <code>paid_at</code>. 3. La orden confirmada pasa a ser contabilizable en los reportes financieros. 4. Solo el administrador puede ejecutar esta acción. 5. Se muestra un diálogo de confirmación antes de ejecutar el cambio. 6. La orden desaparece de la lista de pagos QR pendientes tras la confirmación. 7. El campo <code>paid_at</code> en <code>orders</code> se registra con la marca de tiempo exacta de la confirmación.			

Cuadro 2.32. HU-30: Confirmación de pago QR

HU-31 — Mostrar QR estático de pago al cliente

HU-31	Mostrar QR estático de pago al cliente		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero mostrar el QR del banco o billetera digital al cliente directamente desde la app para agilizar el cobro sin necesidad de un dispositivo adicional.			
Criterios de aceptación 1. El administrador puede subir la imagen del QR de pago desde la configuración del restaurante. 2. El mesero puede abrir el QR de pago en pantalla completa desde el detalle de la orden. 3. La imagen del QR se muestra con alta resolución para facilitar el escaneo del cliente. 4. El QR se muestra solo cuando la orden tiene <code>metodo_pago='qr'</code>. 5. La acción de mostrar el QR no modifica ningún estado de la orden. 6. Si no hay QR configurado, el mesero ve un mensaje indicando que debe configurarlo el admin.			

Cuadro 2.33. HU-31: Visualización del QR de pago

2.5.10 Gastos operativos (admin)

HU-32 — Registrar un gasto operativo

HU-32	Registrar un gasto operativo		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero registrar los gastos del día como insumos y pagos a proveedores para tener el egreso real reflejado en los reportes financieros.			
Criterios de aceptación 1. El formulario incluye campos de tipo de gasto, monto, descripción y fecha. 2. El monto solo acepta valores numéricos positivos con hasta dos decimales. 3. El gasto se guarda en la tabla <code>expenses</code> con el <code>created_by</code> del admin en sesión. 4. Solo el administrador puede insertar gastos, validado por la política RLS <code>admin_insert_expenses</code>. 5. Tras guardar, el gasto aparece de inmediato en el listado de egresos. 6. El campo fecha permite registrar gastos de días anteriores, no solo el día actual.			

Cuadro 2.34. HU-32: Registro de gasto operativo

HU-33 — Listar gastos por rango de fechas

HU-33	Listar gastos por rango de fechas		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero ver los gastos registrados en un período determinado para revisar el historial de egresos y compararlo con los ingresos del mismo periodo.			
Criterios de aceptación 1. La pantalla incluye un selector de fecha desde y fecha hasta. 2. La lista resultante muestra tipo, monto, descripción y fecha de cada gasto. 3. El total acumulado del período es visible al final de la lista. 4. El filtro se aplica sobre el campo fecha de la tabla expenses . 5. La lista se ordena cronológicamente por fecha de gasto de forma descendente. 6. La pantalla muestra un estado vacío si no hay gastos en el período seleccionado.			

Cuadro 2.35. HU-33: Listado de gastos por período

HU-34 — Editar y eliminar un gasto

HU-34	Editar y eliminar un gasto		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero corregir o eliminar un gasto mal registrado para mantener los egresos precisos y que los reportes reflejen la realidad financiera del restaurante.			
Criterios de aceptación 1. El formulario de edición llega pre-poblado con los datos actuales del gasto. 2. Todos los campos del gasto son editables: tipo, monto, descripción y fecha. 3. Se muestra un diálogo de confirmación antes de ejecutar la eliminación. 4. Tras eliminar, el gasto desaparece del listado y el total del período se recalcula. 5. Solo el administrador puede editar o eliminar gastos. 6. Al cancelar la edición, no se persiste ningún cambio en la base de datos.			

Cuadro 2.36. HU-34: Edición y eliminación de gastos

2.5.11 Vista de mesas en tiempo real (mesero)

HU-35 — Ver estado de todas las mesas asignadas

HU-35	Ver estado de todas las mesas asignadas		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero ver el estado actual de cada una de mis mesas en una sola pantalla para saber dónde necesito actuar sin tener que revisar cada orden individualmente.			
Criterios de aceptación 1. La pantalla muestra tarjetas por mesa con los estados: Libre, Pendiente, Listo para entregar y Entregado. 2. El estado de cada mesa se deriva del campo estado de la orden activa asociada. 3. Las mesas sin orden activa se muestran con estado Libre. 4. Esta pantalla es la pantalla principal del rol mesero al iniciar sesión. 5. El componente TableCard representa cada mesa de forma visual diferenciada por estado. 6. El mesero solo ve las mesas que tienen órdenes asignadas a él o mesas del restaurante sin orden activa. 7. La pantalla muestra el número y label de cada mesa.			

Cuadro 2.37. HU-35: Vista de estado de mesas del mesero

HU-36 — Actualizar estado de mesas en tiempo real sin recargar

HU-36	Actualizar estado de mesas en tiempo real sin recargar		
Estado	Completada	Asignado	Equipo
Esfuerzo	4 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero que el estado de mis mesas se actualice automáticamente cuando cambie para no tener que refrescar la pantalla durante el servicio.			
Criterios de aceptación 1. El hook <code>useTables</code> se suscribe al canal <code>orders</code> de Supabase Realtime filtrado por <code>mesero_id</code>. 2. Al recibir un evento de cambio en <code>orders</code>, el estado visual de la mesa afectada se actualiza sin recargar la pantalla. 3. La suscripción se activa al montar el componente y se cancela correctamente al desmontarlo. 4. El estado de la suscripción (conectado/desconectado) se maneja sin exponer errores al usuario en condiciones normales. 5. La actualización en UI ocurre en menos de un segundo tras el evento de base de datos. 6. El hook es reutilizable y no genera suscripciones duplicadas si el componente se re-renderiza.			

Cuadro 2.38. *HU-36: Actualización en tiempo real del estado de mesas***HU-37 — Recibir notificación cuando un pedido está listo**

HU-37	Recibir notificación cuando un pedido está listo		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero recibir una notificación visible en la app cuando mi pedido pasa a estado listo para ir a buscarlo a cocina sin tener que estar revisando constantemente la pantalla.			
Criterios de aceptación 1. Al recibir un evento <code>UPDATE</code> en <code>orders</code> donde <code>estado='lista'</code>, se dispara una alerta visible en pantalla. 2. La alerta muestra el número de mesa del pedido listo. 3. La tarjeta de la mesa correspondiente se resalta visualmente con el estado Listo para entregar. 4. La notificación se muestra incluso si el mesero está en otra pantalla de la app. 5. La alerta se descarta manualmente o desaparece tras un tiempo configurable. 6. Solo se notifica al mesero propietario de la orden, no a todos los meseros.			

Cuadro 2.39. *HU-37: Notificación de pedido listo*

HU-38 — Manejar reconexión automática del canal Realtime

HU-38	Manejar reconexión automática del canal Realtime		
Estado	Completada	Asignado	Equipo
Esfuerzo	5 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero que la app se reconecte automáticamente al canal en tiempo real si pierde la conexión para no perder notificaciones críticas durante el servicio.			
Criterios de aceptación 1. Los eventos <code>CHANNEL_ERROR</code> y <code>TIMED_OUT</code> de Supabase Realtime disparan la lógica de reconexión. 2. La reconexión implementa un mecanismo de backoff exponencial para no saturar el servidor. 3. La UI muestra un indicador visual cuando la conexión Realtime está interrumpida. 4. Cuando se restablece la conexión, el indicador desaparece y la suscripción se reanuda. 5. Tras una reconexión, el estado de las mesas se refresca desde la base de datos para corregir posibles eventos perdidos. 6. La lógica de reconexión está encapsulada en una función <code>scheduleReconnect</code> reutilizable. 7. Los intentos de reconexión se detienen si el componente es desmontado antes de completarse.			

Cuadro 2.40. *HU-38: Reconexión automática al canal Realtime*

2.5.12 Llamada al mesero (cliente → mesero)

HU-39 — Mostrar botón “Llamar al mesero” en la vista pública de mesa

HU-39	Mostrar botón “Llamar al mesero” en la vista pública de mesa		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como cliente en mesa, quiero poder llamar al mesero desde mi celular con un botón para no tener que levantarme ni interrumpir la dinámica del restaurante.			
Criterios de aceptación 1. El botón <code>CallWaiterButton</code> es visible en la ruta <code>/table/[slug]/[tableId]</code>. 2. Al presionarlo, se inserta un registro en <code>table_calls</code> con el <code>table_id</code> y <code>restaurant_id</code> validados desde la ruta pública. 3. La inserción usa el cliente público de Supabase y no requiere cuenta del cliente. 4. El botón muestra un estado de carga mientras se procesa la inserción. 5. Tras llamar exitosamente, el botón muestra una confirmación y se deshabilita temporalmente para evitar llamadas duplicadas. 6. Si la mesa o el restaurante no son válidos, la pantalla muestra error y no permite registrar llamadas. 7. El cliente no puede acceder a funcionalidades internas de mesero, admin ni chef.			

Cuadro 2.41. HU-39: Botón de llamada al mesero desde el cliente

HU-40 — Recibir notificación de llamada de cliente en tiempo real

HU-40	Recibir notificación de llamada de cliente en tiempo real		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero recibir una notificación inmediata cuando un cliente en mi mesa me llama para poder atenderlo rápidamente sin que tenga que esperar.			
Criterios de aceptación 1. El hook <code>useTableCalls</code> se suscribe al canal <code>table_calls</code> escuchando eventos <code>INSERT</code>. 2. Al recibir un evento, se muestra una alerta con el número de la mesa que llama. 3. La mesa correspondiente se resalta visualmente con el estado <code>Cliente llama</code> en la vista de mesas. 4. El mesero recibe la notificación incluso si está en otra pantalla de la app. 5. Solo el mesero con una orden activa en esa mesa recibe la notificación; otros meseros no son alertados. 6. La alerta es descartable manualmente por el mesero.			

Cuadro 2.42. HU-40: Notificación de llamada de cliente en tiempo real

HU-41 — Marcar llamada de cliente como atendida

HU-41	Marcar llamada de cliente como atendida		
Estado	Completada	Asignado	Equipo
Esfuerzo	1 punto	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero poder marcar una llamada de cliente como atendida para que la alerta desaparezca y la mesa vuelva a su estado operativo normal.			
Criterios de aceptación 1. Al marcar como atendida, se ejecuta <code>UPDATE table_calls SET atendida=true, atendida_at=now()</code>. 2. El estado visual <code>Cliente llama</code> de la mesa se limpia en la vista de mesas. 3. La acción está disponible desde la alerta de notificación recibida. 4. La operación solo la puede ejecutar el mesero con la política RLS <code>mesero_table_calls</code> activa. 5. El campo <code>atendida_at</code> queda registrado con la marca de tiempo exacta de la atención. 6. Tras marcar la llamada, la mesa regresa al estado que corresponda según su orden activa.			

Cuadro 2.43. HU-41: Marcado de llamada de cliente como atendida

2.5.13 Carta digital pública (web)

HU-42 — Publicar carta digital accesible por URL sin login

HU-42	Publicar carta digital accesible por URL sin login		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como cliente del restaurante, quiero poder ver el menú escaneando un QR desde mi celular sin instalar ninguna app ni crear una cuenta.			
Criterios de aceptación 1. La ruta <code>/menu/[slug]</code> renderiza en el navegador usando el cliente <code>supabasePublic</code> con la anon key. 2. Solo se muestran ítems con <code>disponible=true</code>, aplicando la política <code>public_menu_read</code>. 3. La página es accesible sin autenticación desde cualquier navegador moderno. 4. No se requiere descarga de ninguna aplicación para visualizar la carta. 5. El slug del restaurante define qué menú se muestra, permitiendo múltiples restaurantes en el mismo sistema. 6. La página carga en menos de 3 segundos en condiciones normales de red. 7. Un slug inexistente muestra una página de error descriptiva, no un error técnico.			

Cuadro 2.44. HU-42: Carta digital pública

HU-43 — Mostrar carta organizada por categorías con imágenes y precios

HU-43	Mostrar carta organizada por categorías con imágenes y precios		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como cliente, quiero ver el menú organizado con fotos y precios para decidir qué pedir de forma rápida y atractiva desde mi celular.			
Criterios de aceptación 1. Los ítems están agrupados por categoría con encabezados visibles. 2. Cada ítem muestra la imagen servida desde el CDN de Supabase Storage. 3. El precio de cada ítem es visible con formato monetario claro. 4. El diseño es responsive y está optimizado para dispositivos móviles. 5. Las imágenes se cargan de forma lazy para no bloquear la visualización inicial del menú. 6. Si un ítem no tiene imagen, se muestra un placeholder coherente con el diseño. 7. La carta es visualmente legible sin zoom adicional en pantallas de 5 pulgadas o más.			

Cuadro 2.45. *HU-43: Presentación visual de la carta digital***HU-44 — Generar y mostrar QR del restaurante para carta pública**

HU-44	Generar y mostrar QR del restaurante para carta pública		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero obtener el QR que apunta a la carta digital de mi restaurante para imprimirlo y colocarlo en la entrada o en las mesas para clientes que quieran consultar el menú.			
Criterios de aceptación 1. El QR se genera apuntando a la URL <code>/menu/[slug]</code> del restaurante. 2. El QR es descargable como imagen desde la pantalla de configuración del restaurante. 3. El QR es escaneable desde cualquier aplicación de cámara estándar en Android e iOS. 4. Si el slug del restaurante cambia, el QR se regenera automáticamente con la nueva URL. 5. El QR se muestra en resolución suficiente para ser impreso sin pixelarse. 6. Solo el administrador puede acceder a la sección de generación de QR.			

Cuadro 2.46. *HU-44: Generación del QR de la carta pública*

2.5.14 Vista pública de cliente en mesa (web)

HU-45 — Abrir vista pública al escanear el QR de la mesa

HU-45	Abrir vista pública al escanear el QR de la mesa		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como cliente en mesa, quiero que al escanear el QR de mi mesa se abra una página pública vinculada a esa mesa sin que tenga que registrarme ni instalar una aplicación.			
Criterios de aceptación 1. Al acceder a <code>/table/[slug]/[tableId]</code>, el sistema valida que el restaurante y la mesa existan. 2. El cliente no necesita ingresar ningún dato ni crear una cuenta para usar la vista. 3. Un <code>tableId</code> inexistente muestra un mensaje de error descriptivo al cliente. 4. La ruta pública no tiene acceso a ninguna funcionalidad de mesero, administrador ni chef. 5. Ejemplo local validado: <code>http://localhost:8081/table/jimmy/b3383250-5517-4772-8bda-549bf27d</code>			

Cuadro 2.47. HU-45: Vista pública del cliente en mesa

HU-46 — Ver menú completo desde la sesión de mesa

HU-46	Ver menú completo desde la sesión de mesa		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como cliente en mesa, quiero ver todo el menú incluyendo platos no disponibles para saber qué hay y qué no hay hoy antes de decidir qué pedir.			
Criterios de aceptación 1. La política <code>cliente_full_menu_read</code> permite la lectura de todos los ítems del menú, incluyendo no disponibles. 2. Los ítems no disponibles se muestran con un indicador visual claro (ej.: tachado o etiqueta Agotado). 3. Los ítems disponibles y no disponibles están diferenciados visualmente de forma inmediata. 4. El menú se organiza por categorías igual que en la carta pública. 5. El cliente no puede modificar ni interactuar con los ítems del menú, solo consultarlos. 6. La vista carga desde la ruta pública de mesa y no requiere inicio de sesión del cliente.			

Cuadro 2.48. HU-46: Vista del menú completo desde la sesión de mesa

HU-47 — Generar QR individual por mesa

HU-47	Generar QR individual por mesa		
Estado	Completada	Asignado	Equipo
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero obtener el QR individual de cada mesa para imprimirlo, laminarlo y colocarlo físicamente en la mesa correspondiente.			
Criterios de aceptación 1. Cada mesa en el listado de mesas muestra su propio QR apuntando a <code>/table/[slug]/[tableId]</code>. 2. El QR es descargable como imagen desde la fila de cada mesa en el listado. 3. El QR identifica unívocamente la mesa mediante su <code>tableId</code> en la URL. 4. El QR es escaneable desde cualquier cámara estándar en dispositivos Android e iOS. 5. Si el <code>tableId</code> o el slug cambian, el QR se regenera con la nueva URL. 6. Solo el administrador puede acceder y descargar los QR de mesas. 7. El QR se muestra en resolución suficiente para ser impreso sin pérdida de calidad.			

Cuadro 2.49. HU-47: Generación de QR por mesa

2.5.15 Reportes financieros (admin)

HU-48 — Crear función SQL `get_report` para ingresos y egresos

HU-48 Crear función SQL <code>get_report</code> para ingresos y egresos			
Estado	Completada	Asignado	Equipo
Esfuerzo	4 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero que los reportes financieros se calculen en el servidor mediante una función PostgreSQL para garantizar precisión, seguridad y rendimiento en cualquier volumen de datos.			
Criterios de aceptación 1. La función <code>get_report(date_from date, date_to date)</code> está creada en la migración <code>003_functions.sql</code>. 2. La función valida que el rol del invocante sea <code>admin</code> antes de ejecutar; lanza excepción si no lo es. 3. La función usa <code>SECURITY DEFINER</code> para acceder a tablas protegidas por RLS. 4. Retorna un objeto JSON con <code>gross_income</code>, <code>total_expenses</code> y <code>net_income</code>. 5. Solo contabiliza órdenes con <code>estado='entregada'</code> y <code>pago_confirmado='confirmed'</code>. 6. Los gastos se filtran por <code>fecha</code> dentro del rango indicado. 7. La función es invocable desde la app mediante <code>supabase.rpc('get_report', {...})</code>. 8. Los valores retornados son correctos cuando no hay datos en el período (retorna ceros, no null).			

Cuadro 2.50. *HU-48: Función SQL para reporte financiero*

HU-49 — Crear función SQL `get_top_items` para ranking de platos

HU-49	Crear función SQL <code>get_top_items</code> para ranking de platos		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero tener una función SQL que devuelva los platos más vendidos por período para poblar el ranking en la aplicación del administrador.			
Criterios de aceptación - La función <code>get_top_items(date_from, date_to, item_limit)</code> está creada en la migración <code>003_functions.sql</code>. - Retorna una tabla con columnas <code>menu_item_id</code>, <code>nombre</code>, <code>total_qty</code> y <code>total_revenue</code>. - Los resultados se ordenan por <code>total_qty</code> de forma descendente. - Solo incluye ítems de órdenes con <code>estado='entregada'</code> y <code>pago_confirmado='confirmed'</code>. - El parámetro <code>item_limit</code> tiene valor por defecto de 10. - La función usa <code>SECURITY DEFINER</code> y debe ser envuelta en un validador de rol externo antes de llamarla. - Es invocable desde la app mediante <code>supabase.rpc('get_top_items', {...})</code>.			

Cuadro 2.51. HU-49: Función SQL para ranking de platos

HU-50 — Pantalla de reporte financiero por período

HU-50	Pantalla de reporte financiero por período		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero ver los ingresos brutos, egresos y neto de un período seleccionado para entender la situación financiera del restaurante de forma rápida y clara.			
Criterios de aceptación - La pantalla incluye un selector de período con opciones: diario, semanal, mensual y rango libre. - Los datos se obtienen invocando <code>supabase.rpc('get_report', {date_from, date_to})</code>. - La pantalla muestra <code>gross_income</code>, <code>total_expenses</code> y <code>net_income</code> con formato monetario claro. - Cuando el <code>net_income</code> es negativo, se resalta visualmente como alerta. - La pantalla muestra un estado de carga mientras se obtienen los datos del servidor. - Solo el administrador puede acceder a esta pantalla; cualquier otro rol es redirigido. - Si no hay datos en el período seleccionado, se muestran ceros con un mensaje explicativo.			

Cuadro 2.52. HU-50: Pantalla de reporte financiero

HU-51 — Pantalla de ranking de platos más vendidos

HU-51	Pantalla de ranking de platos más vendidos		
Estado	Completada	Asignado	Equipo
Esfuerzo	3 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero ver qué platos se vendieron más en un período determinado para tomar decisiones informadas sobre el menú y las operaciones del restaurante.			
Criterios de aceptación 1. Los datos se obtienen invocando <code>supabase.rpc('get_top_items', {date_from, date_to, item_limit})</code>. 2. La lista muestra los platos ordenados por cantidad total vendida de forma descendente. 3. Cada entrada muestra nombre del plato, total de unidades vendidas y revenue generado. 4. La pantalla incluye un selector de período idéntico al de la pantalla de reporte financiero. 5. El límite de ítems mostrados es configurable, con valor por defecto de 10. 6. La pantalla muestra un estado de carga mientras se obtienen los datos del servidor. 7. Si no hay ventas en el período, se muestra un estado vacío descriptivo.			

Cuadro 2.53. HU-51: Pantalla de ranking de platos más vendidos

2.5.16 Historias complementarias implementadas

Las siguientes historias documentan funcionalidades reales agregadas al alcance final del proyecto para reflejar el código implementado en los módulos de cocina, inventario, reportes y cierre operativo.

ID	Historia	Estado	Criterios principales
HU-52	Implementar rol chef/cocina	Completada	El rol chef existe en <code>profiles.rol</code> ; el guard redirige a (chef); cocina tiene navegación propia.
HU-53	Panel de pedidos para cocina	Completada	El chef visualiza órdenes activa y lista; se muestran mesa, hora e ítems; los datos se actualizan con Real-time.
HU-54	Marcar pedido como listo desde cocina	Completada	Cocina actualiza <code>orders.estado='lista'</code> ; el mesero recibe aviso visual; la orden cambia de estado sin recargar.

ID	Historia	Estado	Criterios principales
HU-55	Marcar plato agotado o disponible desde cocina	Completada	El chef alterna <code>menu_items.agotado</code> ; se usa la RPC <code>set_menu_item_sold_out</code> ; el menú refleja el cambio.
HU-56	Crear módulo de inventario de materiales	Completada	El admin accede a <code>inventario</code> ; puede crear/editar/desactivar materiales con unidad y stock mínimo.
HU-57	Registrar reposiciones y consumos de inventario	Completada	El admin registra movimientos <code>reposicion</code> y <code>consumo</code> ; se guardan cantidad, motivo, notas y usuario.
HU-58	Visualizar materiales con stock bajo	Completada	La pantalla calcula el stock actual a partir de movimientos y resalta materiales con stock menor o igual al mínimo.
HU-59	Reporte de ventas por mesero	Completada	La función <code>get_waiter_sales</code> calcula número de órdenes y total vendido por mesero en un rango de fechas.
HU-60	Liberar mesa después de pago confirmado	Completada	La función <code>clear_table_after_payment</code> permite limpiar la mesa cuando la orden entregada tiene pago confirmado.
HU-61	Generar comprobante de orden entregada	Completada	El sistema genera un comprobante HTML con detalle de orden, mesa, ítems, total y método de pago.
HU-62	Registrar fecha efectiva de pago	Completada	Las órdenes confirmadas guardan <code>paid_at</code> ; los reportes se calculan por fecha real de pago.
HU-63	Generar snapshot server-side del estado de mesas	Completada	La RPC <code>get_waiter_table_state_snapshot</code> consolida mesa, orden activa, última orden entregada y llamada pendiente.

Cuadro 2.54. Historias de usuario complementarias implementadas

3. Diseño del Proyecto

3.1 Stack Tecnológico

Capa	Tecnología		Justificación
Mobile y Web	React Native + Expo SDK 54		Permite una misma base de código para Android y rutas web públicas.
Navegación	Expo Router 6		Ruteo basado en archivos con grupos por rol y rutas públicas.
Lenguaje	TypeScript		Tipado estático para reducir errores durante el desarrollo.
Backend	Supabase		PostgreSQL, Auth, Storage, Realtime y funciones SQL administradas.
Base de datos	PostgreSQL		Modelo relacional, integridad referencial, constraints, triggers y RPC.
Autenticación	Supabase	Auth + profiles	Sesiones seguras y perfiles propios con roles admin , mesero y chef .
Seguridad	Row Level Security		Restricción de acceso a datos según rol y usuario autenticado.
Almacenamiento	Supabase Storage		Imágenes del menú y logo del restaurante mediante URLs públicas.
Tiempo real	Supabase Realtime		Sincronización de órdenes, mesas, llamadas e inventario.
Reportes	Funciones PostgreSQL/RPC	PostgreSQL	Cálculos server-side para ventas, egresos, ranking y ventas por mesero.
UI	Componentes propios	MD3	Interfaz consistente para admin, mesero y chef.
Versiones	Git + GitHub		Historial de cambios, ramas auxiliares y respaldo del repositorio.

Cuadro 3.1. *Stack tecnológico real del sistema*

3.2 Modelo de Datos

3.2.1 Entidades y Relaciones

El sistema utiliza una base de datos PostgreSQL administrada por Supabase. El modelo se organiza alrededor de restaurantes, usuarios/perfiles, mesas, menú, órdenes, gastos, llamadas de mesa e inventario.

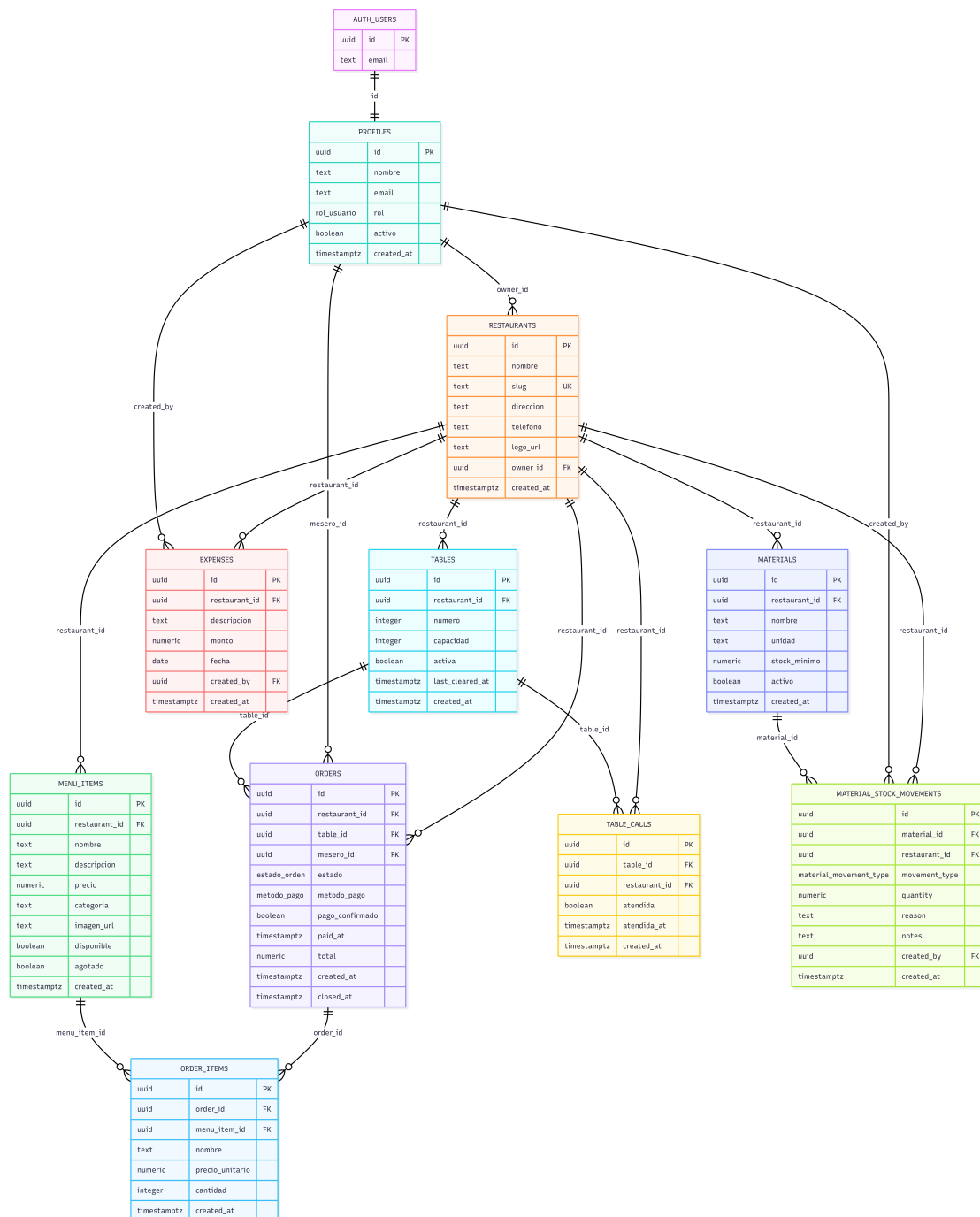


Figura 3.1. Modelo entidad-relación de la base de datos del sistema iComanda

Nota

El archivo `report/erd_mermaid.md` contiene el diagrama ER en formato Mermaid para regenerar la imagen `img/erd_database.png` cuando se actualice el modelo.

3.2.2 Esquema de Tablas Reales

profiles

Campo	Tipo	Descripción
id	uuid	PK, referencia a <code>auth.users</code>
nombre	text	Nombre del usuario
email	text	Correo electrónico
rol	enum	<code>mesero</code> <code>admin</code> <code>chef</code>
activo	boolean	Indica si el usuario puede ingresar
created_at	timestampz	Fecha de creación

restaurants

Campo	Tipo	Descripción
id	uuid	PK
nombre	text	Nombre del restaurante
slug	text UNIQUE	Identificador usado en URLs públicas
direccion	text	Dirección del restaurante
telefono	text	Teléfono de contacto
logo_url	text	Logo público del restaurante
owner_id	uuid FK	Administrador responsable
created_at	timestampz	Fecha de creación

tables

Campo	Tipo	Descripción
id	uuid	PK
restaurant_id	uuid FK	Restaurante al que pertenece
numero	integer	Número de mesa
capacidad	integer	Capacidad aproximada
activa	boolean	Estado activo/inactivo
last_cleared_at	timestampz	Última limpieza/liberación de mesa
created_at	timestampz	Fecha de creación

menu_items

Campo	Tipo	Descripción
id	uuid	PK
restaurant_id	uuid FK	Restaurante propietario
nombre	text	Nombre del plato
descripcion	text	Descripción del plato
precio	numeric(10,2)	Precio actual
categoria	text	Categoría del menú
imagen_url	text	Imagen del plato
disponible	boolean	Visible/disponible para venta
agotado	boolean	Marcado temporal de cocina
created_at	timestampz	Fecha de creación

orders

Campo	Tipo	Descripción
id	uuid	PK
restaurant_id	uuid FK	Restaurante asociado
table_id	uuid FK	Mesa asociada
mesero_id	uuid FK	Mesero que abrió la orden
estado	enum	activa lista entregada cancelada
metodo_pago	enum	efectivo qr tarjeta
pago_confirmado	boolean	Confirmación final del pago
paid_at	timestampz	Fecha efectiva de pago
total	numeric(10,2)	Total de la orden
created_at	timestampz	Fecha de apertura
closed_at	timestampz	Fecha de cierre

order_items

Campo	Tipo	Descripción
id	uuid	PK
order_id	uuid FK	Orden padre
menu_item_id	uuid FK	Plato original, puede quedar nulo
nombre	text	Snapshot del nombre del plato
precio_unitario	numeric(10,2)	Snapshot del precio
cantidad	integer	Cantidad solicitada
created_at	timestampz	Fecha de registro

expenses

Campo	Tipo	Descripción
id	uuid	PK
restaurant_id	uuid FK	Restaurante asociado
descripcion	text	Descripción del gasto
monto	numeric(10,2)	Monto del gasto
fecha	date	Fecha contable
created_by	uuid FK	Usuario que lo registró
created_at	timestampz	Fecha de creación

table_calls

Campo	Tipo	Descripción
id	uuid	PK
table_id	uuid FK	Mesa que solicita atención
restaurant_id	uuid FK	Restaurante asociado
atendida	boolean	Indica si fue atendida
atendida_at	timestampz	Momento de atención
created_at	timestampz	Momento de llamada

materials

Campo	Tipo	Descripción
id	uuid	PK
restaurant_id	uuid FK	Restaurante asociado
nombre	text	Nombre del material o insumo
unidad	text	Unidad de medida
stock_minimo	numeric	Umbral mínimo
activo	boolean	Estado activo/inactivo
created_at	timestampz	Fecha de creación

material_stock_movements

Campo	Tipo	Descripción
id	uuid	PK
material_id	uuid FK	Material afectado
restaurant_id	uuid FK	Restaurante asociado
movement_type	enum	consumo reposicion
quantity	numeric	Cantidad movida
reason	text	Motivo del movimiento
notes	text	Notas adicionales
created_by	uuid FK	Usuario que registró el movimiento
created_at	timestampz	Fecha del movimiento

3.3 Estructura del Proyecto

La estructura real del repositorio sigue las convenciones de Expo Router:

```
iComanda/
|-- app/
|   |-- login.tsx
|   |-- _layout.tsx
|   |-- (admin)/      index, usuarios, mesas, restaurante, menu,
|   |                  pagos-qr, gastos, inventario, reports
|   |-- (mesero)/      index, menu, pedidos, pedido-nuevo, pedido/[id]
|   |-- (chef)/        index
|   |-- menu/[slug].tsx
|   \-- table/[slug]/[tableId].tsx
|-- components/
|   |-- admin/
|   |-- customer/
|   |-- md3/
|   \-- tables/
|-- context/auth.tsx
|-- lib/
|   |-- supabase.ts
|   |-- supabase-public.ts
|   |-- database.types.ts
|   |-- admin.ts
|   |-- order-receipt.ts
```

```
|  \-- hooks/useTables.ts
|-- supabase/
|  |-- migrations/
|  \-- functions/crear-usuario/
|-- report/
|-- app.json
|-- package.json
\-- tsconfig.json
```

3.4 Roles y Accesos

Funcionalidad	Mesero	Admin	Chef
Ver menú disponible	✓	✓	✓
Crear pedidos	✓	—	—
Ver pedidos activos	✓	✓	✓
Marcar pedido como listo	—	—	✓
Marcar pedido entregado/cancelado	✓	✓	—
Vista de mesas en tiempo real	✓	—	—
Atender llamada de cliente	✓	—	—
Gestionar menú	—	✓	—
Marcar plato agotado	—	—	✓
Gestionar mesas y restaurante	—	✓	—
Gestionar usuarios	—	✓	—
Confirmar pagos QR	—	✓	—
Registrar gastos	—	✓	—
Gestionar inventario	—	✓	—
Ver reportes	—	✓	—

Cuadro 3.2. *Matriz real de roles y accesos del sistema*

3.5 Rutas Públicas del Cliente

Además de los roles autenticados, el sistema expone rutas públicas para clientes:

- `/menu/[slug]`: carta digital pública del restaurante. Ejemplo local validado: `/menu/jimmy`.
- `/table/[slug]/[tableId]`: vista pública de mesa para ver menú y llamar al mesero. Ejemplo local validado: `/table/jimmy/b3383250-5517-4772-8bda-549bf27d44e1`.

Estas rutas no requieren una cuenta del cliente y están protegidas por políticas RLS y consultas limitadas al restaurante/mesa correspondiente.

4. Arquitectura del Sistema

La arquitectura del sistema Gestión de Comandas adopta el modelo **BaaS (Backend as a Service)**. En este enfoque, el cliente móvil consume directamente los servicios proporcionados por Supabase sin la necesidad de un backend intermedio personalizado.

Supabase provee servicios administrados de base de datos, autenticación, almacenamiento de archivos y sincronización en tiempo real, lo cual reduce la complejidad de infraestructura y permite enfocar el desarrollo en la lógica de negocio y la experiencia de usuario.

4.1 Diagrama de Contexto

El diagrama de contexto muestra la interacción general entre los actores del sistema (mesero, administrador, chef y cliente) y los servicios principales utilizados en la solución. Se observa cómo la aplicación móvil y la carta digital web se comunican directamente con Supabase mediante protocolos REST, WebSocket y mecanismos de autenticación basados en JWT.

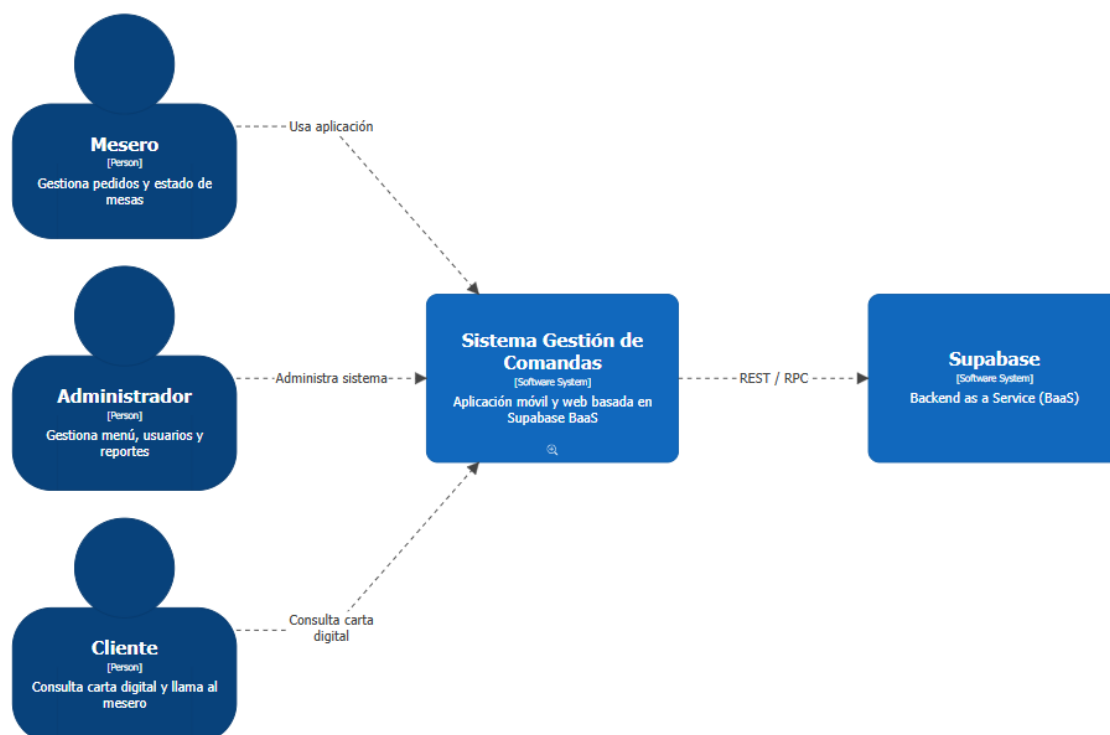


Figura 4.1. Diagrama de Contexto del sistema Gestión de Comandas basado en modelo BaaS

En este nivel se identifican los usuarios del sistema y los sistemas externos con los que interactúa la solución. El cliente móvil implementado en React Native consume directamente los servicios de Supabase PostgreSQL, Supabase Auth, Supabase Storage y Supabase Realtime. Asimismo, la carta digital accesible mediante navegador web permite a los clientes consultar el menú y solicitar atención sin necesidad de autenticación tradicional.

4.2 Diagrama de Contenedores

El diagrama de contenedores describe los bloques principales que componen la solución y cómo estos se comunican entre sí. Se distinguen claramente los contenedores del cliente móvil, las rutas web públicas y los servicios gestionados por Supabase.

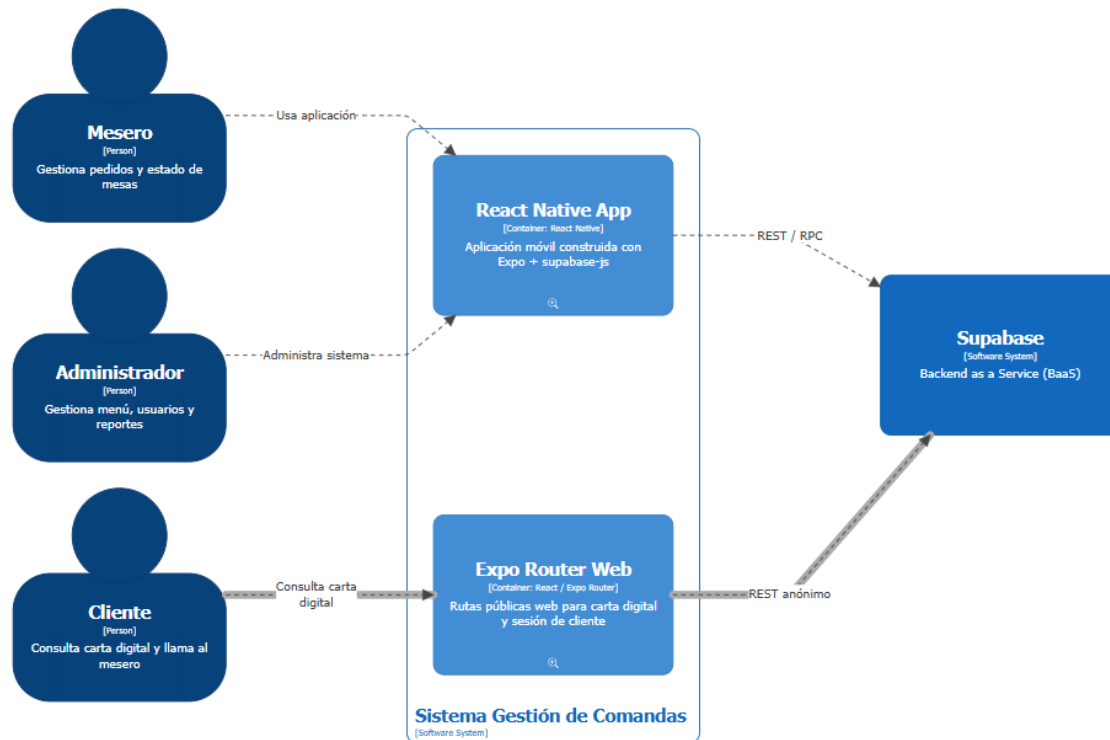


Figura 4.2. Diagrama de Contenedores del sistema utilizando Supabase como Backend as a Service

La aplicación móvil desarrollada con React Native y Expo se conecta directamente a Supabase mediante el SDK oficial supabase-js. Este cliente permite realizar consultas a la base de datos, autenticación de usuarios, subida de archivos y suscripción a eventos en tiempo real.

Por otro lado, Expo Router permite exponer rutas web públicas que facilitan la visualización de la carta digital y la vista pública del cliente en mesa mediante el uso de códigos QR. Estas rutas usan acceso público controlado mediante políticas de seguridad Row Level Security (RLS).

Supabase actúa como proveedor de Backend as a Service ofreciendo:

- Base de datos PostgreSQL con funciones SQL y políticas RLS.
- Servicio de autenticación basado en JWT y roles.
- Almacenamiento de imágenes del menú mediante Storage con acceso público.
- Canales de comunicación en tiempo real mediante WebSocket.

4.3 Tecnologías Utilizadas

La implementación real del proyecto utiliza **Expo SDK 54**, **Expo Router 6**, **React 19**, **React Native 0.81**, **TypeScript** y **supabase-js v2**. A nivel de aplicación se definieron grupos de rutas para (**admin**), (**mesero**) y (**chef**), además de rutas públicas para la carta digital y la sesión de mesa del cliente.

El backend administrado se apoya en Supabase PostgreSQL para persistencia, Supabase Auth para sesiones, Supabase Storage para imágenes, Supabase Realtime para sincronización operativa y funciones RPC para procesos transaccionales como creación de órdenes, reportes, ranking de platos, ventas por mesero, limpieza de mesa e inventario.

4.4 Diagrama de Componentes — Funcionalidades Clave

4.4.1 Componentes de la aplicación móvil

El siguiente diagrama muestra los componentes principales de la aplicación móvil utilizada por el mesero, el administrador y cocina. Estos componentes permiten gestionar pedidos, visualizar el estado de mesas, administrar el menú, controlar inventario, marcar pedidos listos y consultar reportes financieros.

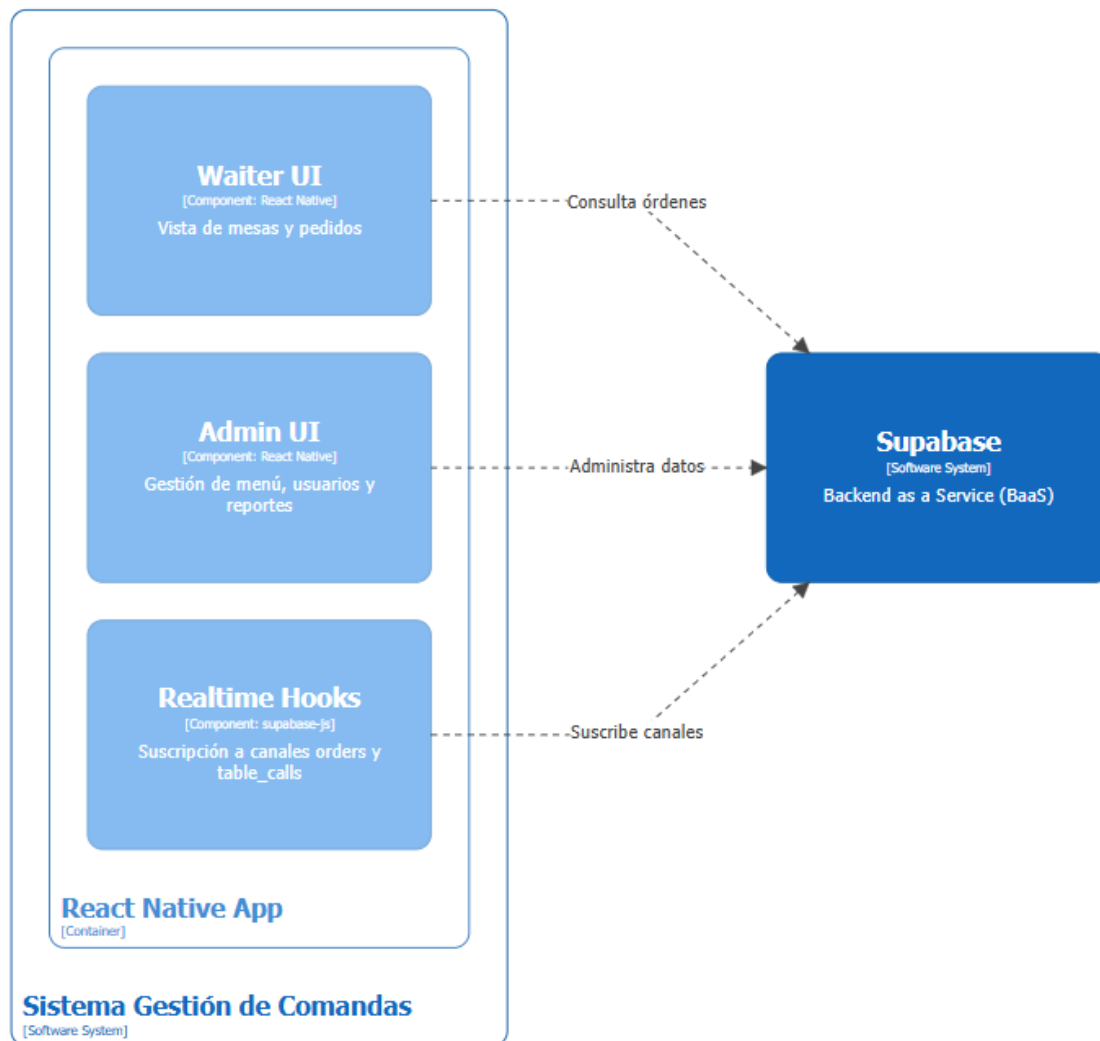


Figura 4.3. Componentes principales de la aplicación React Native

Los componentes Mesero UI, Admin UI y Chef UI representan las interfaces de usuario según el rol autenticado. El módulo Realtime Hooks permite la suscripción a los canales de Supabase Realtime para actualizar el estado de las mesas y los pedidos sin necesidad de recargar la aplicación.

4.4.2 Componentes de rutas web públicas

El siguiente diagrama muestra los componentes asociados a la carta digital y la vista del cliente en mesa. Estas funcionalidades permiten que el cliente visualice el menú mediante un código QR y pueda solicitar atención al mesero.

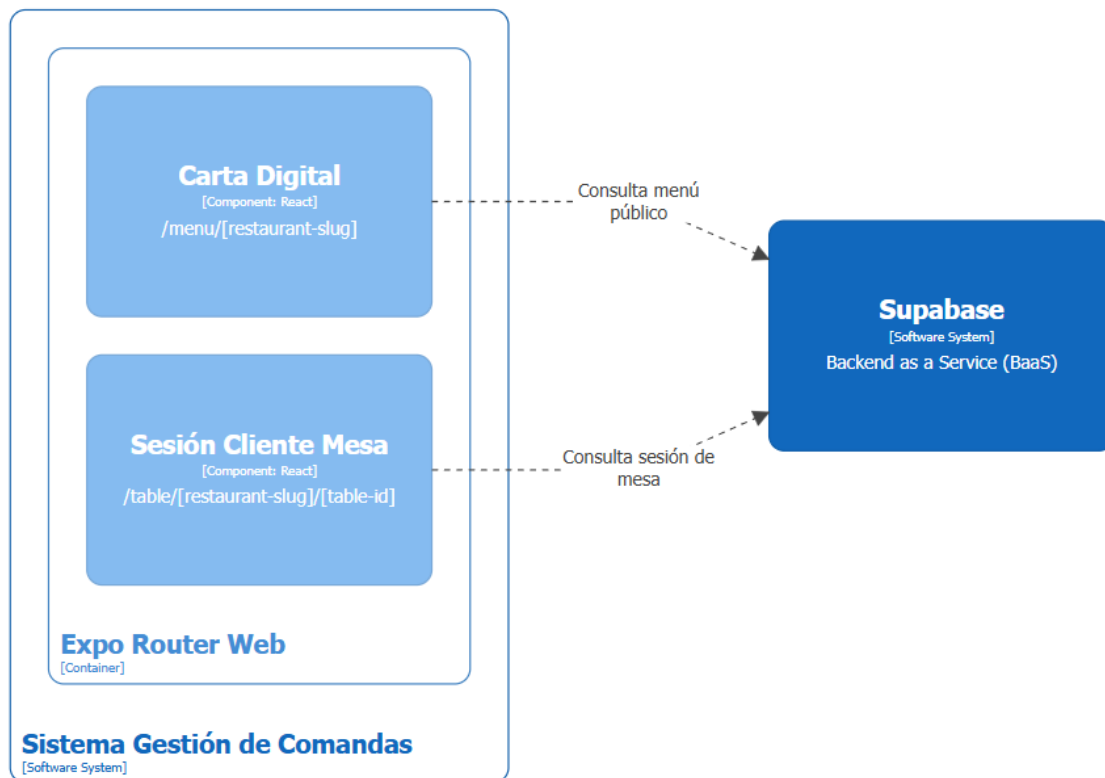


Figura 4.4. Componentes de la carta digital y vista pública de cliente en mesa

Las rutas públicas permiten acceder al menú del restaurante utilizando un identificador único (slug). La sesión de mesa permite identificar la mesa desde la cual el cliente solicita atención, registrando eventos en la base de datos que son posteriormente consumidos por la aplicación del mesero en tiempo real.

4.4.3 Contexto de Supabase

El siguiente diagrama muestra cómo la aplicación cliente interactúa con los servicios ofrecidos por Supabase dentro del modelo Backend as a Service (BaaS). En este enfoque, el cliente móvil y las rutas web públicas consumen directamente los servicios de autenticación, base de datos, almacenamiento y comunicación en tiempo real sin necesidad de un backend intermedio personalizado.



Figura 4.5. Relación entre la aplicación cliente y los servicios de Supabase

Supabase proporciona un conjunto de servicios integrados que permiten reducir la complejidad de infraestructura. El SDK supabase-js facilita la conexión desde la aplicación

React Native hacia PostgreSQL, Auth, Storage y Realtime utilizando protocolos seguros basados en JWT y WebSocket.

4.4.4 Contenedores de Supabase

El diagrama de contenedores presenta los servicios principales que conforman el Backend as a Service utilizado en el sistema. Cada contenedor cumple una función específica dentro de la arquitectura, permitiendo desacoplar responsabilidades y facilitar la escalabilidad del sistema.

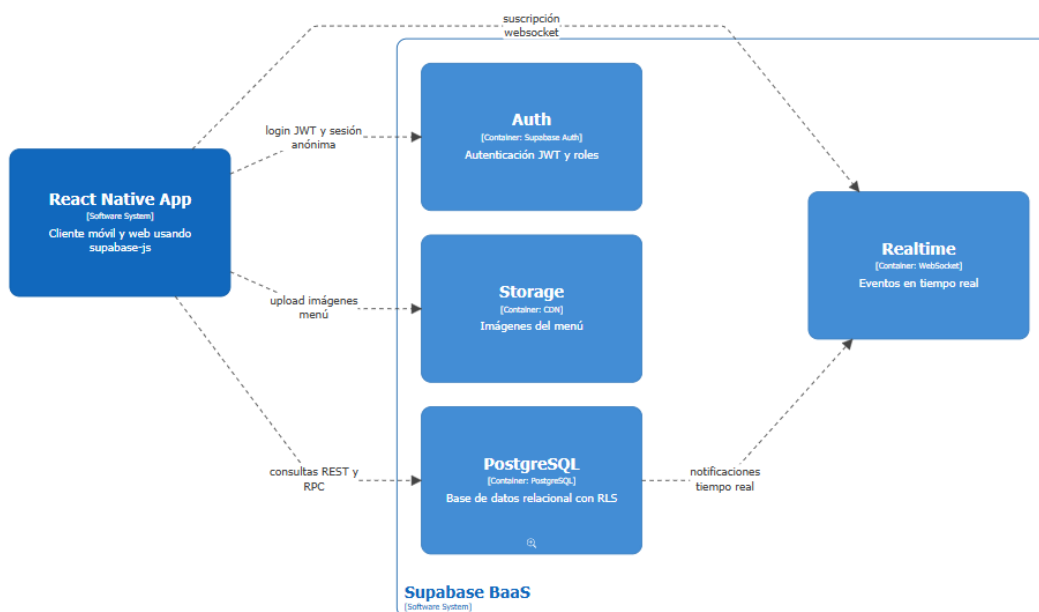


Figura 4.6. Contenedores principales del entorno Supabase

El contenedor PostgreSQL almacena la información relacionada con usuarios, mesas, pedidos, gastos, inventario y reportes. El servicio Auth permite gestionar sesiones de usuario mediante tokens JWT para roles internos. Las rutas de carta digital y mesa se consultan mediante acceso público limitado por RLS.

El servicio Storage permite almacenar imágenes de los productos del menú, las cuales son accesibles mediante URLs públicas optimizadas mediante CDN. Por otro lado, el servicio Realtime permite mantener sincronizados los estados de pedidos y llamadas de clientes mediante WebSockets.

4.4.5 Componentes de la Base de Datos

El siguiente diagrama describe los componentes internos de la base de datos implementada en Supabase. Se incluyen las tablas principales, sus relaciones lógicas y las funciones SQL utilizadas para generar reportes financieros y análisis de ventas.

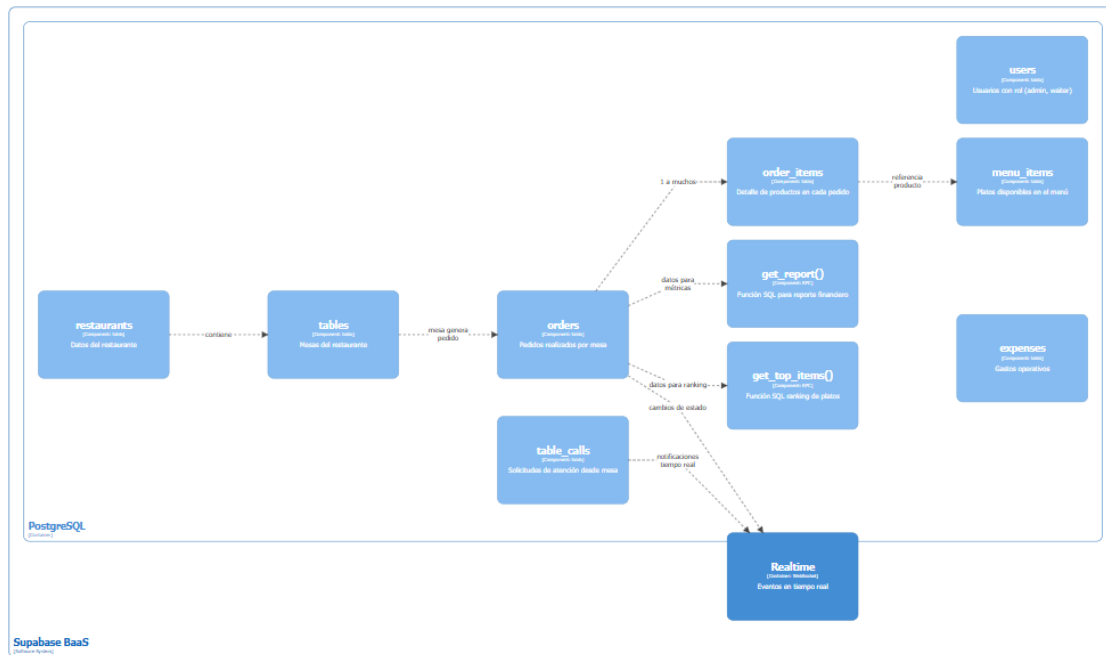


Figura 4.7. Componentes internos del esquema de base de datos en Supabase

La tabla **profiles** almacena los usuarios del sistema con sus respectivos roles. La tabla **restaurants** permite gestionar la información del restaurante, mientras que la tabla **tables** representa las mesas disponibles en el establecimiento.

Las tablas **orders** y **order_items** almacenan la información de los pedidos realizados, manteniendo consistencia histórica de precios mediante snapshots de los productos seleccionados. La tabla **expenses** permite registrar los gastos operativos necesarios para el cálculo de métricas financieras. Las tablas **materials** y **material_stock_movements** permiten controlar materiales, reposiciones, consumos y stock mínimo.

4.4.6 Vista de Estado de Mesas (Mesero)

La pantalla principal del mesero muestra en tiempo real el estado de cada mesa asignada. Esta funcionalidad se basa en la suscripción al canal **orders** de Supabase Realtime, permitiendo actualizar los estados de los pedidos sin necesidad de recargar la aplicación.

Adicionalmente, la aplicación escucha el canal **table_calls**, el cual permite detectar cuando un cliente solicita atención desde su mesa mediante la carta digital.

Estado visual	Condición en DB
Libre	Sin orden activa para esa mesa en la fecha actual
Pendiente	<code>estado = 'activa'</code>
Listo para entregar	<code>estado = 'lista'</code>
Entregado	<code>estado = 'entregada'</code>
Cliente llama	Registro en <code>table_calls</code> con <code>atendida = false</code>

Este mecanismo permite mantener sincronizados los estados entre múltiples dispositivos conectados simultáneamente, asegurando consistencia en la información mostrada al personal del restaurante.

5. Metodología Aplicada

5.1 Metodología de Desarrollo

Para el desarrollo del proyecto se adoptó la metodología ágil **Scrum**, dado que permite una alta flexibilidad y adaptación a los cambios, además de fomentar una comunicación constante entre los miembros del equipo. Scrum se basa en ciclos de trabajo cortos y entregables funcionales (sprints) que permiten un seguimiento continuo del progreso.

El proceso de desarrollo sigue las siguientes fases dentro del marco de Scrum:

- **Product Backlog:** Lista priorizada de los requisitos del proyecto, mantenida y priorizada por el Product Owner.
- **Sprint Planning:** Antes de cada sprint el equipo selecciona los ítems del backlog a completar y se compromete a entregarlos.
- **Daily Stand-Up:** Reuniones diarias breves para revisar progreso, detectar bloqueos y ajustar tareas.
- **Sprint Review:** Al final del sprint se presenta el incremento al Product Owner y partes interesadas.
- **Sprint Retrospective:** El equipo identifica áreas de mejora en el proceso y aplica ajustes al siguiente sprint.

5.2 Ciclo de Vida del Proyecto

El proyecto se organizó en **3 sprints de 2 semanas** cada uno:

Sprint	Alcance
1	Configuración del proyecto Expo, conexión con Supabase, diseño inicial del esquema de base de datos, autenticación, roles, RLS y estructura base de rutas para administrador y mesero.
2	Gestión administrativa del restaurante, usuarios, mesas, menú, registro de pedidos, flujo de estados de orden, pagos, gastos y vista de mesas en tiempo real mediante Supabase Realtime.
3	Carta digital pública por QR, sesión de cliente en mesa, llamada al mesero, rol chef/cocina, control de platos agotados, inventario de materiales, reportes financieros, ranking de platos y pruebas finales.

Cuadro 5.1. *Release Plan — Sprints del proyecto*

5.3 Herramientas de Gestión

- **Notion:** Gestión del sprint backlog con vista Kanban (Por hacer / En progreso / En revisión / Completado), centralización de documentación técnica y seguimiento de tareas.
- **GitHub:** Control de versiones distribuido y respaldo del repositorio. Se utilizaron ramas auxiliares para desarrollo personal, integradas luego a la rama principal y eliminadas después del merge.
- **Discord:** Canal principal de comunicación cotidiana, dividido en espacios temáticos (desarrollo, diseño, pruebas).
- **Excalidraw:** Diagramación colaborativa de esquemas de base de datos, flujos de navegación y arquitectura del sistema.
- **Postman:** Testing y documentación de llamadas RPC y REST durante el desarrollo e integración.

6. Seguimiento y Desarrollo Grupal

6.1 Planificación del Trabajo

Sprint 1 — Fundación: base de datos, autenticación y roles

Objetivo: Setup del proyecto, schema de BD, autenticación, RLS y CRUD del menú.

Historias de Usuario

■ Setup e infraestructura

- HU-01: Inicializar proyecto Expo con TypeScript y Expo Router
- HU-02: Configurar proyecto Supabase y variables de entorno
- HU-03: Crear migración de esquema inicial de la base de datos
- HU-04: Crear constraint de orden activa única por mesa
- HU-05: Crear trigger set_closed_at al entregar una orden

■ Autenticación y roles

- HU-06: Implementar pantalla de login para mesero y admin
- HU-07: Proteger rutas según rol con guards de navegación
- HU-08: Crear trigger sync_role_to_auth para sincronizar rol
- HU-09: Implementar cierre de sesión

■ Row Level Security

- HU-10: Implementar RLS para la tabla orders
- HU-11: Implementar RLS para expenses, menu_items y table_calls

■ Gestión de usuarios (admin)

- HU-12: Listar usuarios del sistema (admin)
- HU-13: Crear nuevo usuario mesero o admin (admin)
- HU-14: Activar y desactivar usuario (admin)

■ Gestión de mesas (admin)

- HU-15: CRUD de mesas del restaurante (admin)
- HU-16: Gestionar restaurante y su slug (admin)

Sprint 2 — Operación: menú, pedidos, pagos y gastos

Objetivo: Registro de pedidos, vista de mesas en tiempo real y módulo financiero.

Historias de Usuario

■ Gestión de menú (admin)

- HU-17: Listar ítems del menú con filtro por categoría (admin)
- HU-18: Crear ítem del menú con imagen (admin)
- HU-19: Editar precio y descripción de un ítem (admin)
- HU-20: Activar o desactivar disponibilidad de un ítem (admin)
- HU-21: Eliminar ítem del menú (admin)

■ Toma de pedidos (mesero)

- HU-22: Ver el menú disponible al tomar un pedido (mesero)
- HU-23: Crear una nueva orden seleccionando mesa e ítems (mesero)
- HU-24: Ver detalle de una orden activa (mesero)
- HU-25: Agregar ítems a una orden existente (mesero)
- HU-26: Marcar una orden como entregada (mesero)
- HU-27: Cancelar una orden (mesero y admin)
- HU-28: Seleccionar método de pago al crear la orden (mesero)

■ Pagos (admin)

- HU-29: Ver órdenes con pago QR pendiente de confirmación (admin)
- HU-30: Confirmar pago QR de una orden (admin)
- HU-31: Mostrar QR estático de pago al cliente (mesero)

■ Gastos operativos (admin)

- HU-32: Registrar un gasto operativo (admin)
- HU-33: Listar gastos por rango de fechas (admin)
- HU-34: Editar y eliminar un gasto (admin)

Sprint 3 — Tiempo real, carta digital, cocina, inventario y reportes

Objetivo: Carta digital pública, sesión de cliente en mesa, rol chef, inventario, reportes y cierre funcional.

Historias de Usuario

■ Vista de mesas en tiempo real (mesero)

- HU-35: Ver estado de todas las mesas asignadas (mesero)

- HU-36: Actualizar estado de mesas en tiempo real sin recargar (mesero)
- HU-37: Recibir notificación cuando un pedido está listo (mesero)
- HU-38: Manejar reconexión automática del canal Realtime (mesero)
- **Llamada al mesero (cliente → mesero)**
 - HU-39: Mostrar botón "Llamar al mesero.^{en} la sesión de cliente en mesa
 - HU-40: Recibir notificación de llamada de cliente en tiempo real (mesero)
 - HU-41: Marcar llamada de cliente como atendida (mesero)
- **Carta digital pública (web)**
 - HU-42: Publicar carta digital accesible por URL sin login
 - HU-43: Mostrar carta organizada por categorías con imágenes y precios
 - HU-44: Generar y mostrar QR del restaurante para carta pública (admin)
- **Vista pública de cliente en mesa (web)**
 - HU-45: Abrir vista pública al escanear el QR de la mesa
 - HU-46: Ver menú completo (incluyendo no disponibles) desde la sesión de mesa
 - HU-47: Generar QR individual por mesa (admin)
- **Reportes financieros (admin)**
 - HU-48: Crear función SQL `get_report` para ingresos y egresos
 - HU-49: Crear función SQL `get_top_items` para ranking de platos
 - HU-50: Pantalla de reporte financiero por período (admin)
 - HU-51: Pantalla de ranking de platos más vendidos (admin)
- **Cocina y disponibilidad operativa**
 - HU-52: Implementar rol chef/cocina
 - HU-53: Panel de pedidos para cocina
 - HU-54: Marcar pedido como listo desde cocina
 - HU-55: Marcar plato como agotado o disponible desde cocina
- **Inventario y control de materiales**
 - HU-56: Crear módulo de inventario de materiales
 - HU-57: Registrar reposiciones y consumos de inventario
 - HU-58: Visualizar materiales con stock bajo
- **Reportes y cierre operativo**
 - HU-59: Reporte de ventas por mesero

- HU-60: Liberar mesa después de pago confirmado
- HU-61: Generar comprobante de orden entregada
- HU-62: Registrar fecha efectiva de pago con `paid_at`
- HU-63: Generar snapshot server-side del estado de mesas

63 historias de usuario · 3 sprints · 6 semanas

6.2 Roles y Responsabilidades

Integrante	Rol
Acebey Zalazar Jimmy Randall	Desarrollador
Cáceres Véliz Erick Alvaro	Desarrollador
Revollo Bernal Amilcar Diego	Scrum Master
Siles Espicho Adrian Eloy	Desarrollador
Tinta Mamani Jairo Amado	Desarrollador

6.3 Control de Versiones

El proyecto utiliza un sistema de control de versiones distribuido con **Git**, gestionado en un único repositorio alojado en GitHub (**iComanda**). Este enfoque centralizado permite una coordinación eficiente entre el desarrollo del frontend móvil, las rutas web públicas y las migraciones de base de datos.

La estrategia de ramas adoptada es:

- **main:** rama principal del proyecto, utilizada para consolidar los avances estables.
- **Ramas auxiliares:** ramas temporales para desarrollo personal o pruebas de módulos, integradas posteriormente a **main** y eliminadas después del merge.

Nota

Para evidenciar el control de versiones en la entrega final, se recomienda incluir una captura del repositorio en GitHub mostrando el historial de commits o la vista general del repositorio, además de un QR que redirija al repositorio. Nombres usados: `repositorio-captura.png` y `qr-repo.png`. URL del repositorio: <https://github.com/DiegoBernalDev/iComanda.git>.

6.4 Reporte de Avances

6.4.1 Indicadores de Progreso

ID	Historia de Usuario	Completada
HU-01	Inicializar proyecto Expo con TypeScript y Expo Router	✓
HU-02	Configurar proyecto Supabase y variables de entorno	✓
HU-03	Crear migración de esquema inicial de la base de datos	✓
HU-04	Crear constraint de orden activa única por mesa	✓
HU-05	Crear trigger set_closed_at al entregar una orden	✓
HU-06	Implementar pantalla de login para mesero y admin	✓
HU-07	Proteger rutas según rol con guards de navegación	✓
HU-08	Crear trigger sync_role_to_auth para sincronizar rol	✓
HU-09	Implementar cierre de sesión	✓
HU-10	Implementar RLS para la tabla orders	✓
HU-11	Implementar RLS para expenses, menu_items y table_calls	✓
HU-12	Listar usuarios del sistema (admin)	✓
HU-13	Crear nuevo usuario mesero o admin (admin)	✓
HU-14	Activar y desactivar usuario (admin)	✓
HU-15	CRUD de mesas del restaurante (admin)	✓
HU-16	Gestionar restaurante y su slug (admin)	✓
HU-17	Listar ítems del menú con filtro por categoría (admin)	✓
HU-18	Crear ítem del menú con imagen (admin)	✓
HU-19	Editar precio y descripción de un ítem (admin)	✓
HU-20	Activar o desactivar disponibilidad de un ítem (admin)	✓
HU-21	Eliminar ítem del menú (admin)	✓
HU-22	Ver el menú disponible al tomar un pedido (mesero)	✓
HU-23	Crear una nueva orden seleccionando mesa e ítems (mesero)	✓
HU-24	Ver detalle de una orden activa (mesero)	✓
HU-25	Agregar ítems a una orden existente (mesero)	✓
HU-26	Marcar una orden como entregada (mesero)	✓
HU-27	Cancelar una orden (mesero y admin)	✓
HU-28	Seleccionar método de pago al crear la orden (mesero)	✓
HU-29	Ver órdenes con pago QR pendiente de confirmación (admin)	✓
HU-30	Confirmar pago QR de una orden (admin)	✓
HU-31	Mostrar QR estático de pago al cliente (mesero)	✓
HU-32	Registrar un gasto operativo (admin)	✓
HU-33	Listar gastos por rango de fechas (admin)	✓
HU-34	Editar y eliminar un gasto (admin)	✓
HU-35	Ver estado de todas las mesas asignadas (mesero)	✓
HU-36	Actualizar estado de mesas en tiempo real sin recargar (mesero)	✓

ID	Historia de Usuario	Completada
HU-37	Recibir notificación cuando un pedido está listo (mesero)	✓
HU-38	Manejar reconexión automática del canal Realtime (mesero)	✓
HU-39	Mostrar botón "Llamar al mesero. ^{en} la sesión de cliente en mesa	✓
HU-40	Recibir notificación de llamada de cliente en tiempo real (mesero)	✓
HU-41	Marcar llamada de cliente como atendida (mesero)	✓
HU-42	Publicar carta digital accesible por URL sin login	✓
HU-43	Mostrar carta organizada por categorías con imágenes y precios	✓
HU-44	Generar y mostrar QR del restaurante para carta pública (admin)	✓
HU-45	Abrir vista pública al escanear el QR de la mesa	✓
HU-46	Ver menú completo (incluyendo no disponibles) desde la sesión de mesa	✓
HU-47	Generar QR individual por mesa (admin)	✓
HU-48	Crear función SQL get_report para ingresos y egresos	✓
HU-49	Crear función SQL get_top_items para ranking de platos	✓
HU-50	Pantalla de reporte financiero por período (admin)	✓
HU-51	Pantalla de ranking de platos más vendidos (admin)	✓
HU-52	Implementar rol chef/cocina	✓
HU-53	Panel de pedidos para cocina	✓
HU-54	Marcar pedido como listo desde cocina	✓
HU-55	Marcar plato como agotado o disponible desde cocina	✓
HU-56	Crear módulo de inventario de materiales	✓
HU-57	Registrar reposiciones y consumos de inventario	✓
HU-58	Visualizar materiales con stock bajo	✓
HU-59	Reporte de ventas por mesero	✓
HU-60	Liberar mesa después de pago confirmado	✓
HU-61	Generar comprobante de orden entregada	✓
HU-62	Registrar fecha efectiva de pago con paid_at	✓
HU-63	Generar snapshot server-side del estado de mesas	✓

Cuadro 6.1. Indicadores de progreso por historia de usuario

6.4.2 Dificultades Encontradas

- **Pausa automática del free tier de Supabase:** La base de datos se pausa tras períodos de inactividad, por lo que se implementó un ping periódico para mantener activo el servicio durante el desarrollo.
- **Reconexión de canales Realtime:** Las conexiones móviles inestables pueden

interrumpir la sincronización en tiempo real. Se implementó manejo de eventos `CHANNEL_ERROR` y `TIMED_OUT` con reintento automático.

- **Persistencia de precios históricos:** Para mantener la consistencia financiera, el precio del producto se almacena como snapshot en el momento del pedido, evitando inconsistencias cuando el admin modifica el menú.

6.5 Comunicación

El equipo empleó las siguientes herramientas de comunicación y colaboración:

- **Notion:** planificación de tareas, backlog y documentación técnica.
- **GitHub:** control de versiones, respaldo del repositorio y seguimiento del historial de commits.
- **Discord:** canal principal de comunicación durante el desarrollo.
- **WhatsApp:** coordinación rápida para decisiones puntuales.

7. Pruebas y Resultados

7.1 Plan de Pruebas

Las pruebas del sistema se realizaron sobre los flujos principales de iComanda, validando los roles **admin**, **mesero**, **chef** y las rutas públicas del cliente. La verificación fue funcional y exploratoria, revisando que cada módulo cargue correctamente, permita completar sus acciones principales y se integre con Supabase para lectura/escritura de datos.

ID	Rol	Caso	Resultado esperado	Estado
CP-01	General	Inicio de sesión	El usuario ingresa y es redirigido según su rol.	Superado
CP-02	Admin	Gestión operativa	Se accede a usuarios, mesas, menú, pagos, gastos, reportes e inventario.	Superado
CP-03	Mesero	Operación de pedidos	Se visualizan mesas, se crea pedido y se revisa detalle de orden.	Superado
CP-04	Chef	Cocina	Se visualizan pedidos y se controla disponibilidad de platos.	Superado
CP-05	Cliente	Carta y mesa pública	El cliente accede por URL/QR, ve el menú y puede llamar al mesero.	Superado
CP-06	Gestión	Repositorio	Se evidencia control de versiones mediante GitHub y QR del repositorio.	Superado

7.2 Evidencia de Interfaces y Resultados

7.2.1 Acceso al Sistema

La pantalla de inicio de sesión valida el acceso de usuarios internos. Desde este punto, el sistema redirige al módulo correspondiente según el rol cargado desde el perfil del usuario.



Figura 7.1. *Pantalla de inicio de sesión del sistema iComanda.*

7.2.2 Flujo de Administrador

El administrador concentra la configuración operativa del restaurante. Las capturas muestran el panel principal y la gestión del menú, diferenciando el acceso general a módulos de la administración específica de platos.

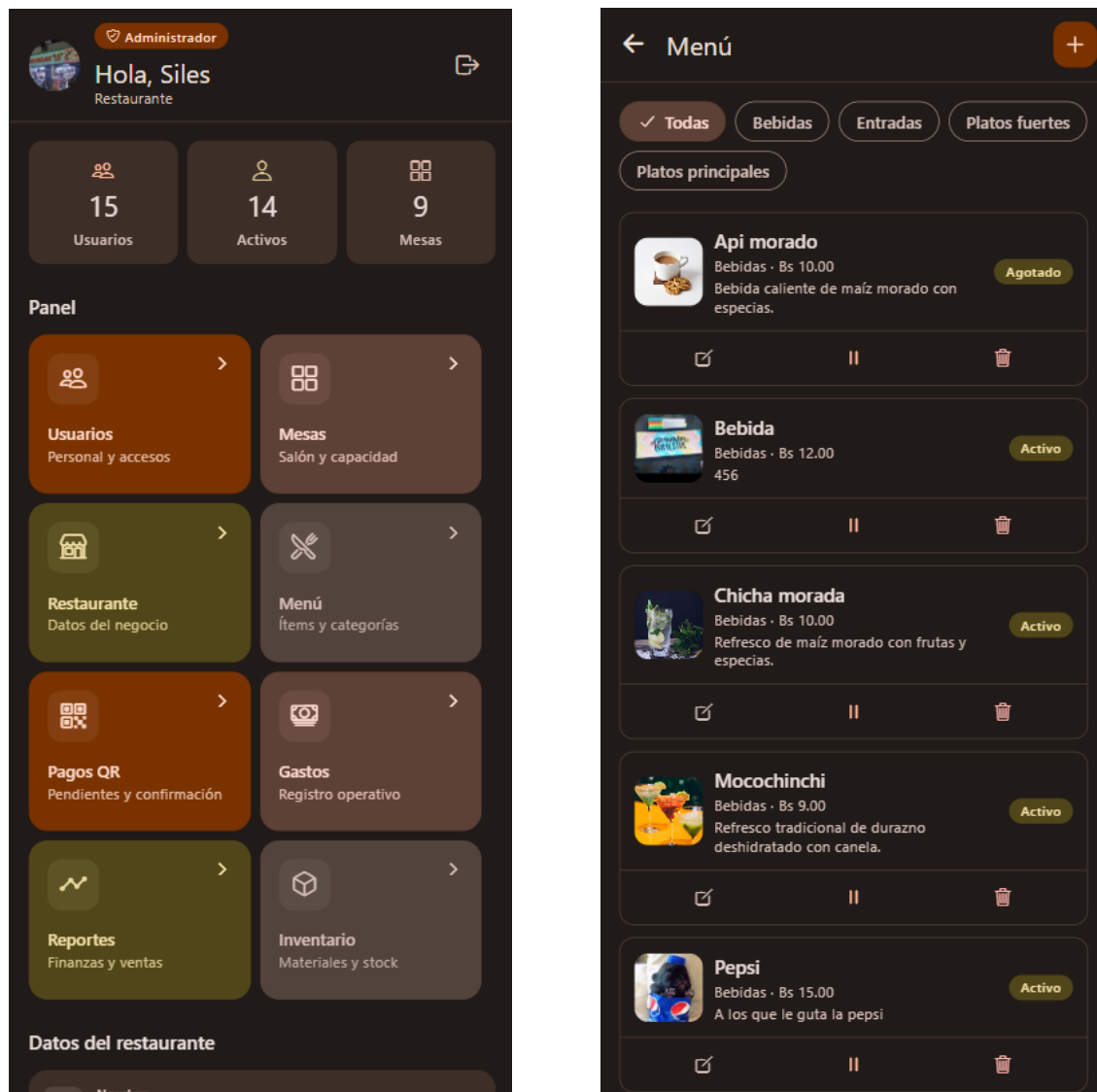


Figura 7.2. Panel principal del administrador y módulo de gestión de menú.

La administración también contempla usuarios, mesas y pagos QR. Estos módulos permiten controlar el personal, la estructura física del restaurante y la validación manual de pagos realizados mediante QR.

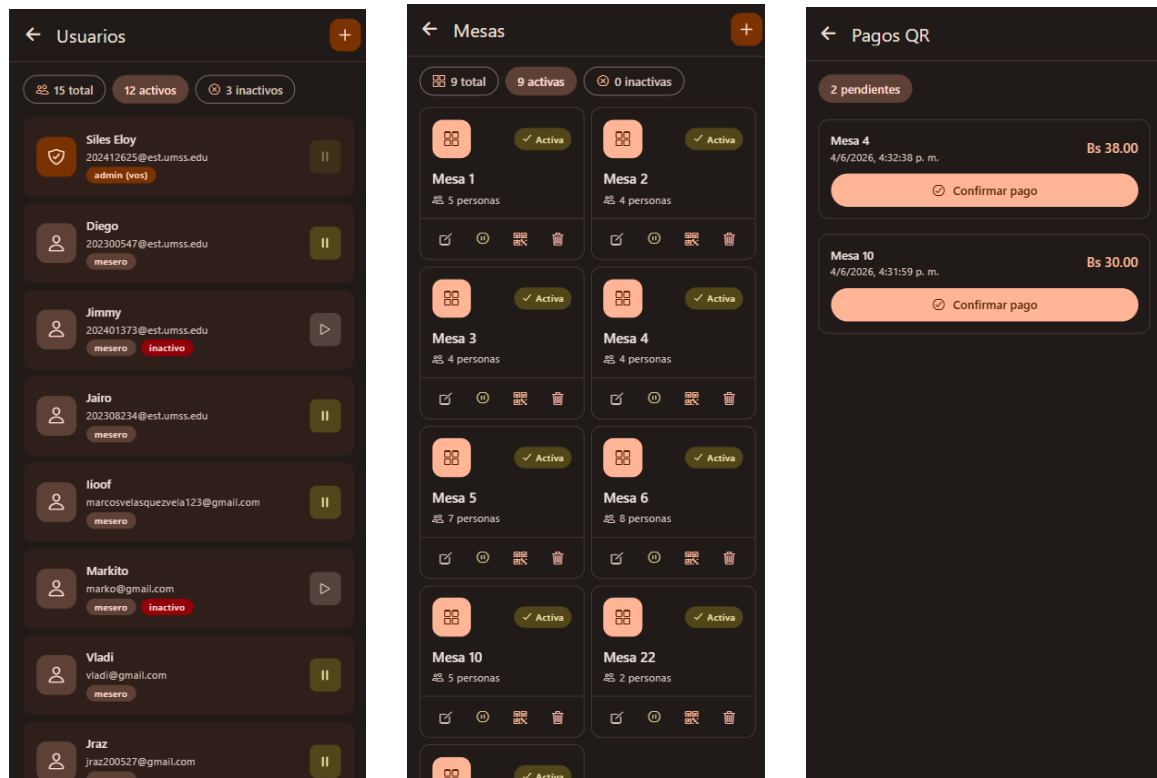


Figura 7.3. Gestión administrativa de usuarios, mesas y pagos QR.

Los módulos financieros y de inventario permiten registrar egresos, analizar ventas, consultar platos más vendidos y controlar materiales del restaurante.

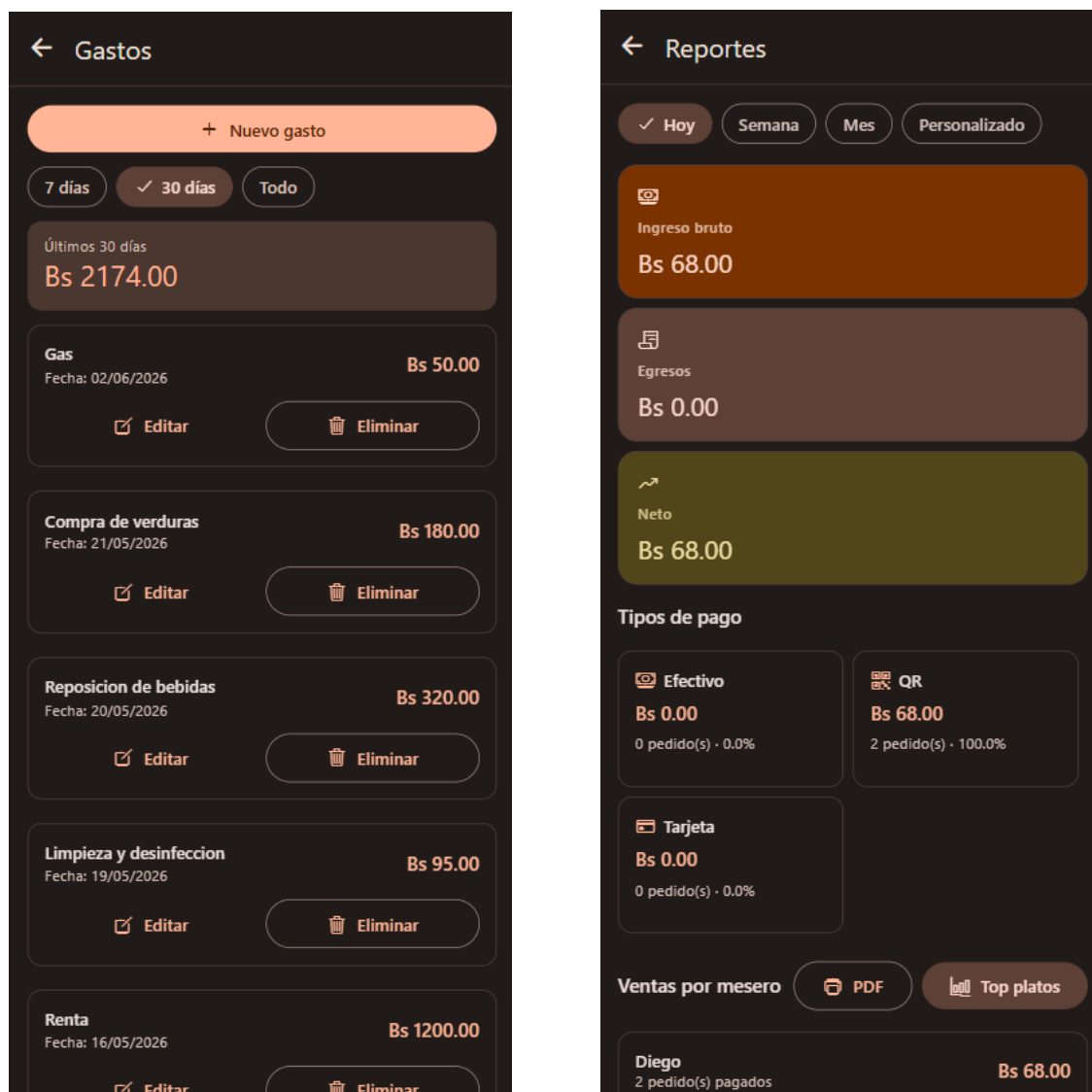


Figura 7.4. Registro de gastos y visualización de reportes financieros.

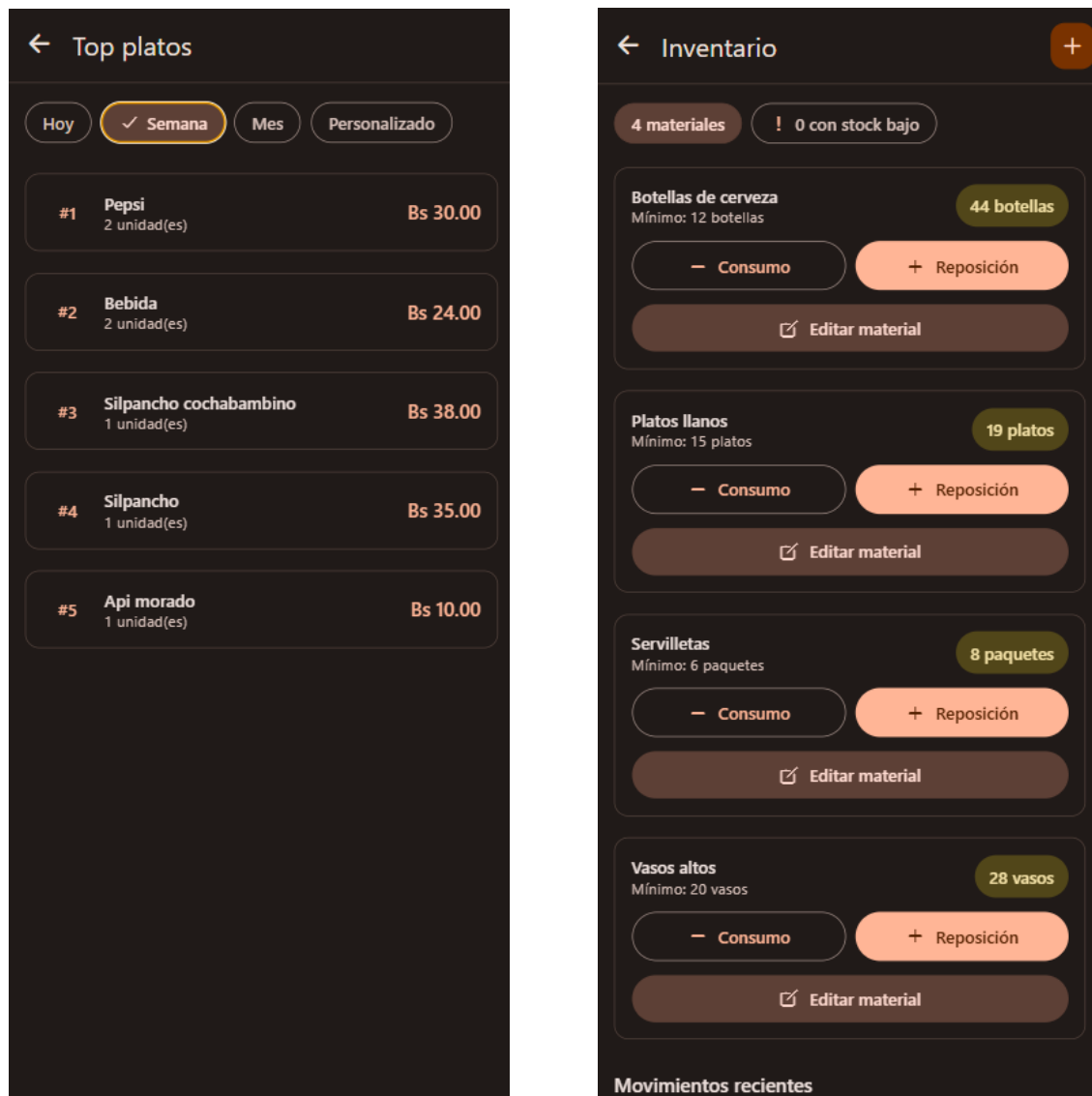


Figura 7.5. *Ranking de platos más vendidos e inventario de materiales.*

7.2.3 Flujo de Mesero

El flujo de mesero valida la operación diaria: ver el estado de mesas, crear pedidos y consultar el detalle de una orden. La vista de mesas se actualiza con los estados operativos como libre, pendiente, listo, pendiente de pago, listo para limpiar o cliente llama.

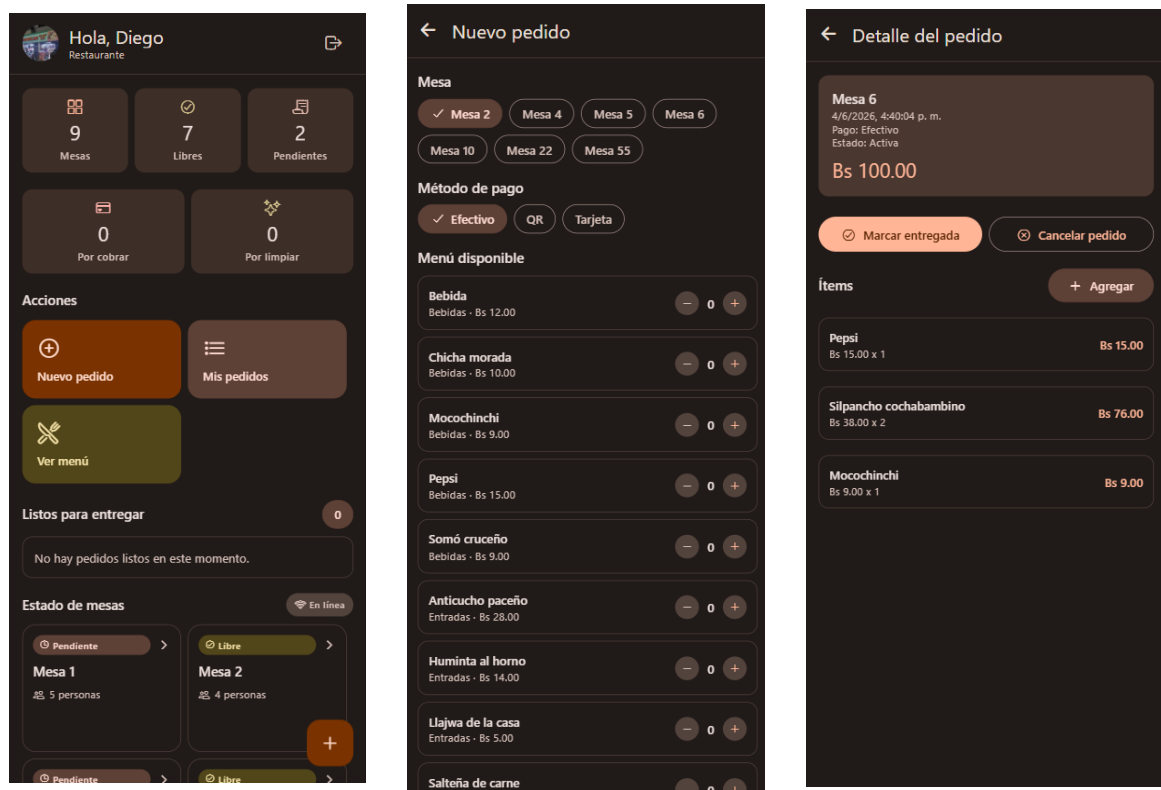


Figura 7.6. Vista de mesas, creación de pedido y detalle de orden del rol mesero.

7.2.4 Flujo de Cocina

El rol chef permite separar la operación de cocina del trabajo del mesero. Desde esta interfaz se revisan pedidos en preparación, se marcan como listos y se controla si un plato está agotado o disponible.

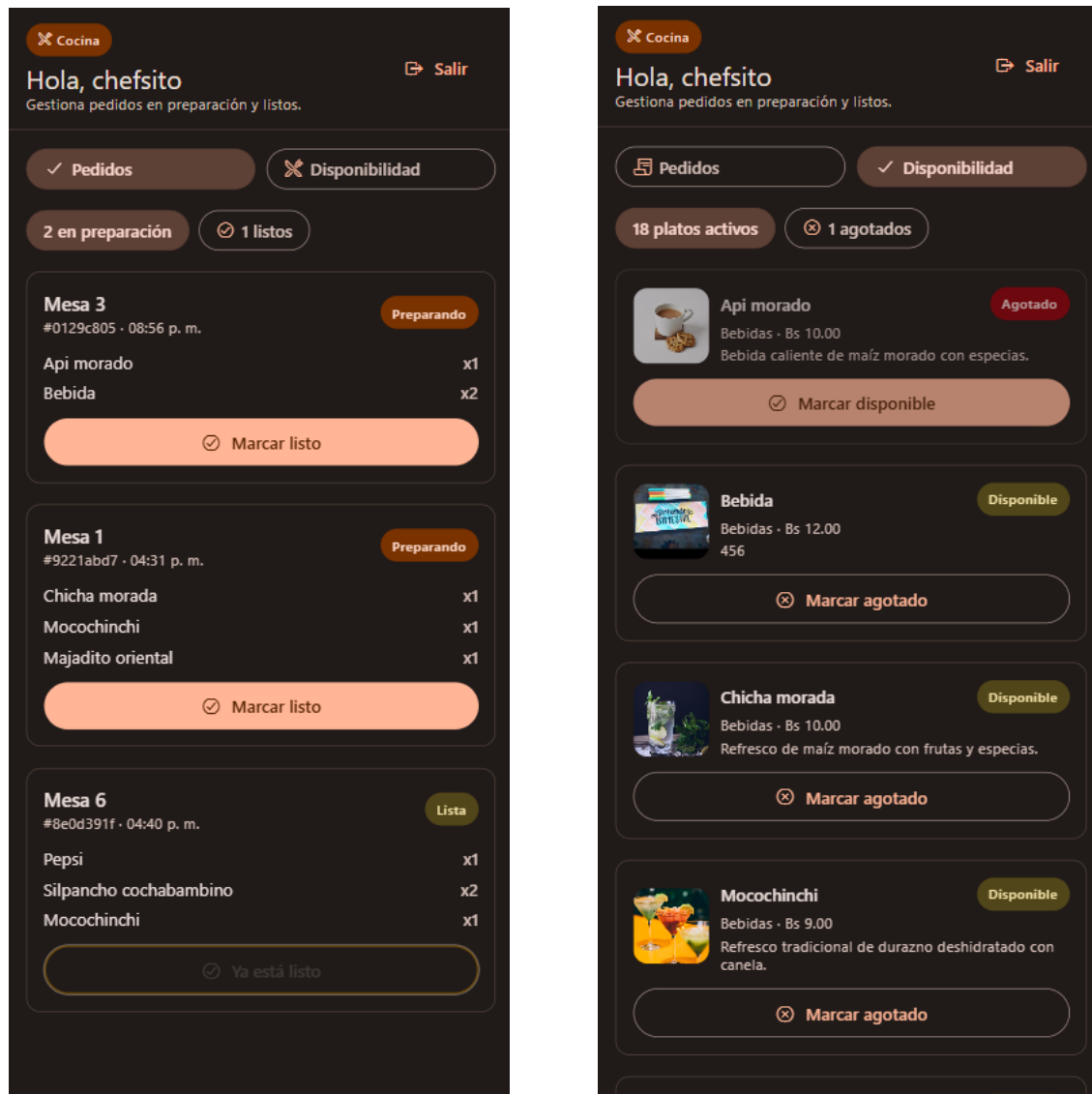


Figura 7.7. Panel de cocina y control de disponibilidad de platos.

7.2.5 Flujo de Cliente

Las rutas públicas permiten que el cliente consulte la carta digital o acceda a la vista de mesa mediante QR. La carta pública validada usa la ruta local `/menu/jimmy`; la vista de mesa permite ver el menú y llamar al mesero sin crear una cuenta.

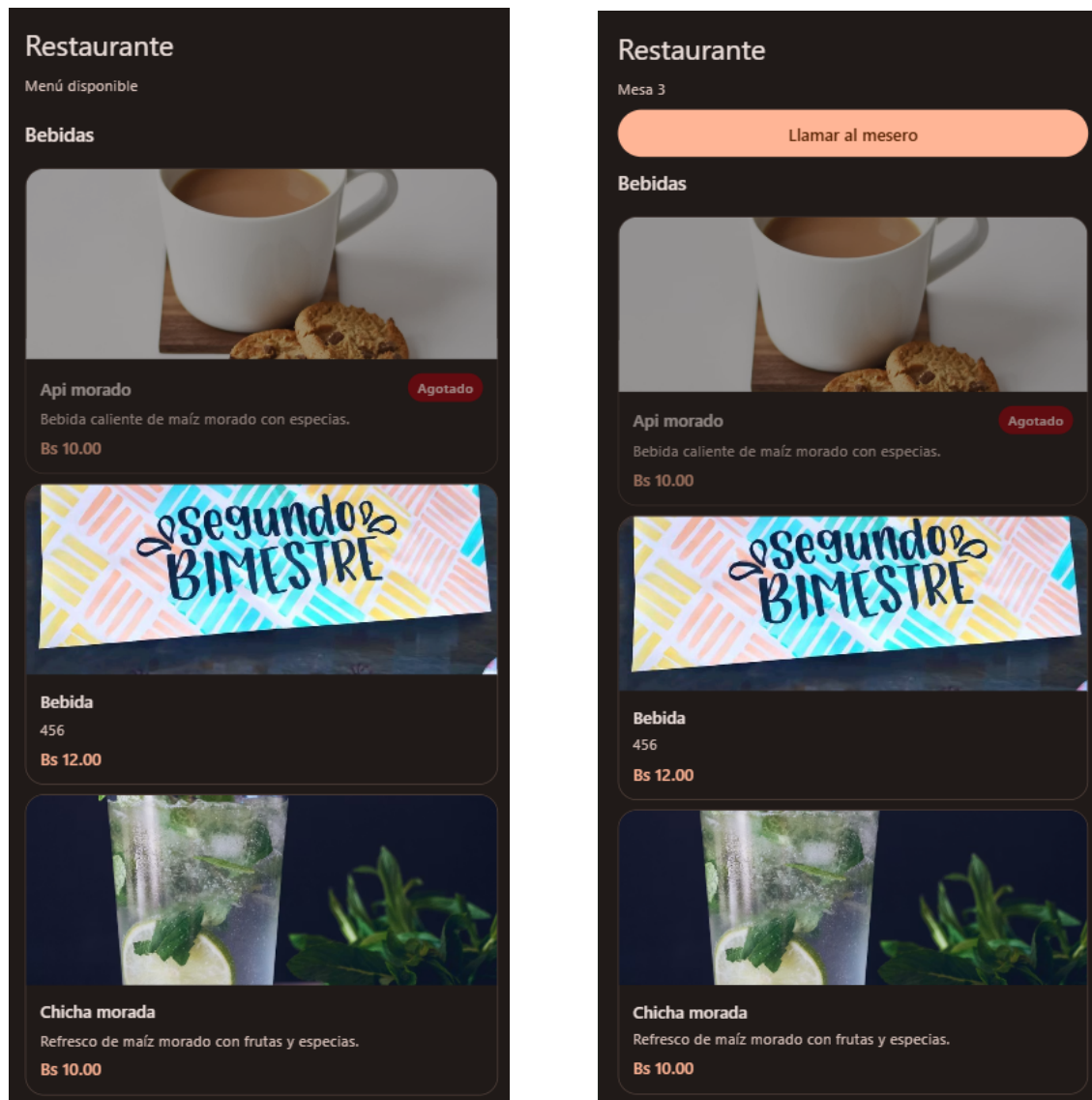


Figura 7.8. Carta pública del restaurante y vista pública de mesa con llamada al mesero.

7.2.6 Control de Versiones

El repositorio del proyecto se encuentra alojado en GitHub. La captura evidencia la existencia del repositorio y el QR facilita el acceso directo al código fuente.

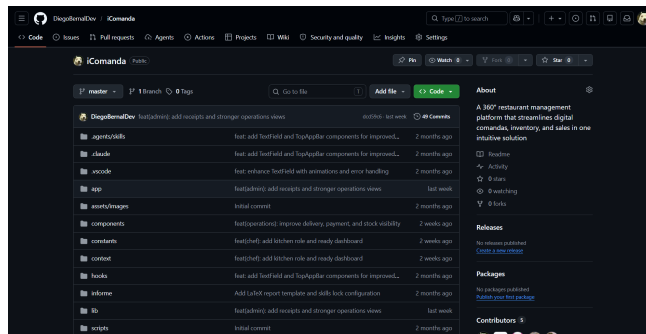


Figura 7.9. Repositorio del proyecto en GitHub y QR de acceso: <https://github.com/DiegoBernalDev/iComanda.git>.

7.3 Resultados Obtenidos

Las pruebas funcionales permitieron validar que los roles principales acceden a sus módulos correspondientes y que las rutas públicas del cliente funcionan sin inicio de sesión. También se verificó la integración con Supabase para lectura de menú, registro de llamadas de mesa, gestión de órdenes, reportes e inventario.

7.4 Mejoras Propuestas

1. Integrar una pasarela de pagos para automatizar la confirmación de QR.
2. Incorporar pruebas automatizadas unitarias y end-to-end.
3. Añadir modo offline básico para pedidos activos.
4. Mejorar analítica de inventario con alertas y proyección de consumo.
5. Agregar monitoreo de errores en producción con una herramienta especializada.

8. Conclusiones

El desarrollo de **iComanda** permitió construir una solución funcional para modernizar la operación de restaurantes pequeños y medianos. El proyecto logró centralizar procesos que normalmente se manejan de forma manual, como la toma de pedidos, el seguimiento de mesas, la comunicación con cocina, la confirmación de pagos, el registro de gastos, el control básico de inventario y la generación de reportes administrativos.

La arquitectura basada en **Supabase como Backend as a Service** resultó adecuada para el alcance del proyecto, ya que permitió utilizar PostgreSQL, Auth, Storage, Realtime, políticas RLS y funciones RPC sin desarrollar un servidor propio. Esta decisión redujo la complejidad de infraestructura y permitió enfocar el esfuerzo en la experiencia móvil, las reglas de negocio y la integración entre roles.

La aplicación implementa tres roles internos principales: **admin**, **mesero** y **chef**. El administrador gestiona la configuración del restaurante, usuarios, mesas, menú, pagos, gastos, inventario y reportes. El mesero opera pedidos y mesas durante el servicio. El chef visualiza pedidos activos, marca órdenes listas y controla platos agotados. Además, los clientes pueden acceder a la carta digital y a la sesión de mesa mediante QR, sin instalar una aplicación.

El trabajo con Scrum en **3 sprints de 2 semanas** ayudó a ordenar el desarrollo en incrementos: primero la base técnica y autenticación, luego la operación principal de pedidos y administración, y finalmente los módulos complementarios de tiempo real, cocina, inventario, carta digital y reportes. Esta división permitió avanzar de forma iterativa y cerrar una versión funcional para la entrega final.

8.1 Alcance Logrado

- Registro y seguimiento de órdenes con estados **activa**, **lista**, **entregada** y **cancelada**.
- Gestión administrativa de usuarios, restaurante, mesas y menú.
- Flujo de cocina con rol **chef** y marcado de platos agotados.
- Carta digital pública y sesión de cliente por mesa mediante QR.
- Llamada del cliente al mesero mediante registros en **table_calls**.
- Confirmación de pagos QR y registro de fecha efectiva de pago.
- Registro de gastos, reportes financieros, ranking de platos y ventas por mesero.
- Módulo de inventario con materiales, reposiciones, consumos y stock bajo.

8.2 Limitaciones Identificadas

- La verificación de pagos QR todavía es manual; no existe integración con una pasarela bancaria o billetera digital.

- El cliente puede consultar menú y llamar al mesero, pero no crea órdenes directamente desde la mesa.
- El sistema depende de conectividad para operar en tiempo real; no se implementó un modo offline completo.
- Las pruebas realizadas se plantean principalmente como manuales y exploratorias; queda pendiente incorporar pruebas automatizadas.
- El uso del plan gratuito de Supabase puede generar pausas por inactividad y límites de recursos si se despliega en producción real.

8.3 Recomendaciones

- Integrar pagos automáticos con una pasarela compatible con el contexto local.
- Añadir pruebas automatizadas con Jest y pruebas end-to-end para flujos críticos.
- Implementar caché local para permitir operación parcial sin conexión.
- Mejorar el módulo de inventario con alertas, historial avanzado y proyección de consumo.
- Preparar un despliegue productivo con monitoreo de errores, backups y un plan de Supabase que evite pausas automáticas.
- Mantener actualizada la documentación del esquema de base de datos, políticas RLS y decisiones de arquitectura conforme evolucione el sistema.

9. Bibliografía

9.1 Tecnologías y Frameworks

- Supabase. *Supabase Documentation*. <https://supabase.com/docs>
- Supabase. *supabase-js v2 Reference*. <https://supabase.com/docs/reference/javascript>
- Supabase. *Row Level Security*. <https://supabase.com/docs/guides/database/postgres/row-level-security>
- Supabase. *Realtime*. <https://supabase.com/docs/guides/realtime>
- Supabase. *Storage*. <https://supabase.com/docs/guides/storage>
- Expo. *Expo Documentation*. <https://docs.expo.dev/>
- Expo. *Expo Router*. <https://docs.expo.dev/router/introduction/>
- React Native. *React Native Documentation*. <https://reactnative.dev/docs/getting-started>
- PostgreSQL Global Development Group. *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>

9.2 Herramientas de Desarrollo y Gestión

- Git. *Git Documentation*. <https://git-scm.com/doc>
- GitHub. *GitHub Documentation*. <https://docs.github.com>
- Notion. *Notion Help Center*. <https://www.notion.com/help>
- Excalidraw. *Excalidraw*. <https://excalidraw.com/>
- Mermaid. *Mermaid Documentation*. <https://mermaid.js.org/>

9.3 Referencias Institucionales

- Universidad Mayor de San Simón. *Universidad Mayor de San Simón*. <https://www.umss.edu.bo/>